

ОБЗОР СОДЕРЖАНИЯ ДИСЦИПЛИНЫ «ОПЕРАЦИОННЫЕ СИСТЕМЫ»

1. Обсуждение функций и эксплуатационных требований к ОС.
2. Процессы, критические участки.
3. Синхронизация процессов, активное и пассивное ожидание разблокирования, аппаратные и программные решения для синхронизации, сложные условия разблокирования, мониторы.
4. Тупики, борьба с тупиками, предупреждение и обход тупиков, обнаружение и восстановление после тупика.
5. Распределение времени процессора в однопроцессорной системе, состояния процесса, многоуровневое планирование.
6. Управление памятью, методы динамического распределения памяти.
7. Виртуальная (многоуровневая) память, стратегии организации списков свободной памяти и вытеснения объектов приложений из памяти, уплотнение памяти.
8. Управление внешней памятью, стратегии оптимизации доступа к объектам во внешней памяти.
9. Организация в/в и файловые системы.
10. Принципы оценки производительности вычислительной системы, тесты, цели исследования производительности, направления использования результатов оценки.
11. Механизмы защиты объектов ОС от окружения.
12. Принципы организации многопроцессорных вычислительных систем и мультипроцессорных ОС, модели программирования для мультипроцессоров.
13. Планирование времени мультипроцессора для связанных и несвязанных процессов.
14. Коммуникационные средства многомашинных вычислительных систем (MPP).
15. Методы реализации взаимодействия распределенных вычислительных систем (ВС, объединенных в компьютерную сеть).
16. Методы реализации когерентности многоуровневой памяти, модели состоятельности памяти, протоколы когерентности (MESI, DASH).
17. Средства виртуализации вычислительной среды.

Литература

1. Олифер В.Г., Олифер Н.А. Сетевые операционные системы. – СПб.: Питер, 2001. – 544 с.

2. Танненбаум Э. Современные операционные системы. – СПб.: Питер, 2002. – 1040 с.
3. Столлингс В. Операционные системы. Внутреннее устройство и принципы проектирования. М.: Издательский дом «Вильямс», 2002. – 848 с.

ЛЕКЦИЯ 1. ОС – Функции и эксплуатационные требования

Вычислительная среда (вычислительная система) = аппаратура вычислителя + ОС.

ОС – преобразователь, переводчик с языка возможностей аппаратуры на язык требований пользователя.

Аппаратура: Центральный процессор, оперативная память, устройства В/В, каналы В/В, групповые устройства управления ВнУ.

Ресурс: объект, распределяемый внутри системы или работающий в режиме разделения несколькими приложениями (процессор, память, УВВ, программы, данные).

Ресурсы:

– **вычислительные, программные, информационные, коммуникационные.**

Различают 2 способа управления разделением ресурсов:

Децентрализованный=взаимодействующие приложения знают о существовании друг друга и взаимодействуют, передавая ресурс друг другу (используется для встроенных систем).

Централизованный = распределение ресурсов возлагается на отдельную программу, которая выделяет ресурс по запросу приложений (используется для систем общего назначения).

Основные функции ОС.

1. Обеспечение интерфейса между аппаратурой и прикладными программами (API)/
2. Предоставление средств для работы с большими объектами информации – файлами.
3. Распределение времени процессора.
4. Управление оперативной памятью.
5. Обеспечение средствами виртуальной памяти.
6. Управление вводом-выводом.
7. Разделение программных ресурсов (организация совместного использования в вычислительной системе одного экземпляра программы одновременно множеством пользователей).

Ключевая цель ОС как многомерного объекта: эффективное использование ресурсов вычислительной системы при хорошем (справедливом) обслуживании задач – это ограничение.

Независимо от полноты реализации функций, ОС должна удовлетворять **эксплуатационным требованиям.**

1. Надежность – надежность ОС должна быть не ниже надежности аппаратуры, ОС должна обнаружить аппаратную ошибку и исправить ее, либо свести к минимуму ущерб от ошибки.
2. Защита – ОС должна защищать пользователей от случайных ошибок других пользователей и от вмешательства злоумышленника.

3. Эффективность – ОС должна быть не требовательна к ресурсам (занимать минимум ресурсов) и управлять ресурсами вычислителя так, чтобы минимизировать простой ресурсов (обеспечить максимально возможную загрузку ресурсов).
4. Удобство – ОС должна быть гибкой и удобной для пользования.
5. Предсказуемость качества обслуживания (справедливость распределения ресурсов вычислителя между пользователями) – ОС должна обеспечивать ожидаемые операционные характеристики.

В системах общего назначения равновесие между требованиями пользователей, техническими ограничениями и функциональностью обеспечивается централизованным распределением ресурсов системы.

Традиционный подход при проектировании ОС состоит в том, что множество процессов, выполняющих основные функции, подчиняются главной программе – супервизору. Супервизор осуществляет централизованное управление и связывает воедино все части системы.

Функции супервизора:

1. Контроль и управление выполнением приложений.
2. Организация связи между функциональными подсистемами ОС.
3. Защита объектов ОС и приложений от случайного и злонамеренного вторжения. Реализация супервизорного и пользовательского режимов работы вычислителя.
4. Выполнение обслуживающих функций, обладающих универсализмом при умеренном потреблении ресурсов.

Супервизор реализует централизацию ключевых функций управления.

Преимущества:

Простота организации – все ключевые функции сосредоточены в одном месте, а не рассеяны по всей ОС.

Простота реализации управления и защиты – просто отделяются части системы друг от друга и пользователей.

Недостатки:

Супервизор становится узким местом при интенсивном взаимодействии различных программ.

Трудность реконструкции (с целью повышения производительности) и внесения изменений в функциональность т.к. супервизор объединяет взаимозависимые функции.

ЛЕКЦИЯ 2. ОС – Процессы

Компоненты вычислительной системы (аппаратура) обычно работают параллельно. ОС выполняет множество вычислительных работ, которые продвигаются вперед почти независимо.

Понятие процесса – это формализация идеи независимой работы. Процесс является единицей работы, выполняемой вычислителем. Поскольку работы независимы, то представляющие их процессы независимы. Поскольку аппаратура работает с различными скоростями, то допускаются произвольные соотношения скоростей выполнения процессов.

Свойства процессов. Процессы – независимые работы, выполняются параллельно, выполняются с различными скоростями. Параллельно выполняющимся работам необходимо обмениваться информацией при совместном использовании ресурсов вычислителя.

Процесс – область памяти, содержимое которой меняется по правилам, описываемой программой, которую интерпретирует ЦП.

Процесс – пространство состояний набора переменных состояния $X=\{x_1, x_2, \dots, x_k\}$; функция действия в этом пространстве; особые элементы пространства состояний – начальные состояния.

Состояние – описывается заданием значений всех переменных состояний.

Переменные состояния – ячейки памяти.

Действие – присвоение значений некоторым переменным набора X .

Функция действия – функция, которая отображает состояния в действия (процедура).

Реализация процессов. В реальной вычислительной системе процесс это выполнение процедуры на ЦП. Каждый процесс однозначно определяется информационной структурой – дескриптором:

1. Идентификатор процесса.
2. Переменная состояния, определяющая положение процесса (готов к работе, работающий, заблокирован).
3. Защищенная область памяти (область сохранения), в которой хранятся текущие значения регистров при прерывании процесса.
4. Информация о ресурсах, которыми владеет процесс.
5. Разделяемые переменные, необходимые для общения с другими процессами.

В вычислителе процесс представляется дескриптором и областью памяти, содержащей программный код и обрабатываемые данные – программный процесс.

В ОС без структурной организации все процессы равноправны и порождаются обычно супервизором.

В ОС со структурной организацией процессов все процессы как правило организованы в виде древовидной структуры. Имеются предки и потомки. Распределение ресурсов находится под контролем: ресурсы любого процесса были когда-то собственностью его предка. Одни процессы имеют больше власти, чем другие – предок управляет потомком.

Состояния процесса: заблокированный, готов, выполняемый.

ЛЕКЦИЯ 3. Синхронизация процессов

Цель ОС – эффективное использование аппаратных и программных ресурсов вычислительной среды.

ТОГДА: ресурсы необходимо разделять между процессами, каждый момент времени аппаратные (физические) и программные (логические) ресурсы доступны только одному процессу.

КРИТИЧЕСКИЙ РЕСУРС – ресурс, допускающий обслуживание только 1 пользователя (процесса) в каждый момент времени. Процессы, использующие критический ресурс должны синхронизировать свои действия. Разделяемые переменные – пример критического ресурса.

КРИТИЧЕСКИЙ УЧАСТОК ПРОЦЕССА – участок, в котором происходит обращение к критическим ресурсам. Критические участки должны быть взаимно исключаемыми.

Механизм синхронизации процессов механизм недопущения одновременного выполнения критических участков – должен обладать 2-мя свойствами:

1. Если ≥ 1 процесса хотят войти в критический участок, то 1 из них должен получить разрешение войти в свой критический участок.
2. В каждый момент только 1 процессу разрешается находиться в своем критическом участке.

Синхронизация с помощью приемов нижнего уровня.

1. Блокировка памяти (операции над памятью – неделимы) = Алгоритм Деккера (для синхронизации 2-х процессов), трудно обобщается на произвольное число процессов.
2. Неделимая операция ПРОВЕРИТЬ и УСТАНОВИТЬ (ПиУ) = Test and Set Lock (TSL). Операция имеет два параметра ОБЩ (глобальная переменная разделяемая процессами), ЛОК (локальная переменная процесса) и выполняется в виде двух присвоений:

ПиУ(ЛОК,ОБЩ)==ЛОК:=ОБЩ;

ОБЩ:=1;

Значения переменной ОБЩ: 1 – блокирующее, 0 – разблокирующее.

begin int ОБЩ; ОБЩ:=0;

parbegin

ПРОЦЕСС1:

begin int ЛОК1; ЛОК1:=1;

do while (ЛОК1=1)

```

ПиУ(ЛОК1,ОБЩ);

end;

<критический участок>

ОБЩ:=0;

<оставшаяся часть процесса1>;

end;

```

ПРОЦЕСС2:

```

begin int ЛОК2; ЛОК2:=1;

do while (ЛОК2=1)

ПиУ(ЛОК2,ОБЩ);

end;

<критический участок>

ОБЩ:=0;

<оставшаяся часть процесса2>;

end;

parend; end;

```

Основной недостаток приемов нижнего уровня – **активное ожидание** входа в критический участок. На основе ПиУ можно построить МЬЮТЕКС – переменная, которая может находиться в 2-х состояниях – заблокированное и неблокированное. Инструмент, реализующий пассивное ожидание – семафоры.

СЕМАФОРЫ

Семафор – средство, позволяющее организовать пассивное ожидание (реализуется обычно в супервизоре).

Семафор **S** – целочисленная переменная, значения которой могут изменять неделимые (для прикладных процессов) операции **P(S)** и **V(S)**

Различают **семафоры общие** и **двоичные**

Операция **P(S)** – **попросить** ресурс, связанный с семафором **S**:

```

begin

S:=S-1;

if S>=0 then <процесс продолжает работать>

```


else <процесс останавливается и встает в очередь ожидания, связанную с семафором S , процесс остается заблокированным до тех пор пока операция $V(S)$, выполненная другим процессом не разблокирует его>;

end;

Операция $V(S)$ – вернуть ресурс, связанный с семафором S :

begin

$S:=S+1$;

if $S>0$ then <процесс продолжает работать>

else <один процесс удаляется из очереди ожидания, получает ресурс и разрешение продолжить работу; продолжает работать либо процесс, который обратился в операции $V(S)$, либо процесс, получивший ресурс>;

end;

Область значений общего семафора S :

$-Pr \leq S \leq Res$,

Pr = количество конкурирующих процессов,

Res = количество единиц ресурса.

При $S=-$ количество конкурирующих процессов: имеем ТУПИК!

Область значений двоичного семафора S : $-1 \leq S \leq +1$

$S=+1$ – ни один из процессов не находится в своем критическом участке,

$S=0$ – один из процессов находится в своем критическом участке,

$S=-1$ – один из процессов находится в своем критическом участке и один процесс – в очереди в семафоре S .

Задача **ПОСТАВЩИК – ПОТРЕБИТЕЛЬ** решается с помощью семафоров

begin int БУФ, \$общий семафор – текущее количество свободных буферов\$

ЗАНЯТО, \$общий семафор – текущее количество занятых буферов\$

СИНХРО; \$двоичный семафор – гарантирует, что в каждый момент времени только 1 процесс работает с буферными стрелками \$

БУФ:=<исходное количество свободных буферов в системе>;

ЗАНЯТО:=0<исходное количество занятых буферов в системе>;

parbegin

ПОСТАВЩИК:

begin

do while (true)

<приготовить сообщение>

P(БУФ); P(СИНХРО);

<послать сообщение>

V(ЗАНЯТО); V(СИНХРО);

end; end;

ПОТРЕБИТЕЛЬ:

begin

do while (true)

P(ЗАНЯТО); P(СИНХРО);

<получить сообщение>;

V(БУФ); V(СИНХРО);

<обработать сообщение>;

end; end;

parend; end;

Семафоры не всегда удобны и приводят к сложным не прозрачным конструкциям в случае необходимости организации совместной работы с множеством типов ресурсов и обработкой набора событий.

Монитор управления взаимодействием параллельных процессов:

1. Разделяемая управляющая структура данных (централизация управляющей информации)
2. Набор функций доступа к структуре данных для управления синхронизацией и коммуникациями между процессами (функции – неделимые операции для приложений=реализованы в ядре).

+: 1. Избавляет программиста от деталей реализации работы с разделяемыми данными;

2. Используется для организации сложных ожиданий (совокупности ресурсов и событий, одного из ресурсов или события из некоторого множества).

Пример реализации: **Монитор на основе таблицы синхронизации (ТС)**

Принципы:

1. Синхронизация взаимодействующих процессов осуществляется через анализ и последующее изменение полей **ТС**.
2. В **ТС** представлены все синхронизируемые процессы, упорядоченные по убыванию приоритета.
3. Наименьший приоритет имеет вспомогательный процесс, постоянно находящийся в состоянии готовности. Получает управление при блокировании всех взаимодействующих процессов (ТУПИКе) и устраняет тупиковую ситуацию.
4. В строке, соответствующей каждому процессу содержится: а) адрес области сохранения; б) переменная состояния процесса (готов/не готов); в) тайм-аут ожидания выделения ресурсов или наступления событий; г) ссылка на список ресурсов, один из которых ожидает процесс; д) события, наступление которых может привести к переключению процессов.

Процесс	Область сохр	Готовность	Таймер	Ресурсы	Событие1	Событие2	...	СобытиеN
P1	Обл1	Не готов	0	Список1	0	0		0
P2	Обл2	Не готов	5	Список2	0	1		0
P3	Обл3	Готов	0	0		0		0
P4	Обл4	Не готов	9	0	1	1		1
P5	Обл5	Готов	0	0	0	0		0
P6	Обл6	Готов	0	0	0	0	0	0

Функции управления и синхронизации процессов реализуются с помощью процедур монитора: **TP**, **TV**, **POST**, **WAIT**, которые анализируют и изменяют поля **ТС**. Номер строки текущего процесса присваивается переменной монитора **ТЕКУЩИЙ**. Текущим может стать только самый приоритетный процесс из всех готовых к работе.

Логика операций монитора.

Операцией **TP** процесс пользуется при необходимости получения критических ресурсов, локализованных внутри тела монитора. Обращаясь к монитору процесс, может указать через параметры: а) несколько семафоров с различными видами ресурсов, одним из которых процесс хотел бы обладать; б) набор событий, наступления одного из которых ожидает процесс; в) тайм-аут ожидания одного из ресурсов или событий. Процесс задерживается на время поиска запрошенных ресурсов. При наличии хотя бы одного запрошенного вида ресурса (идентифицируемого семафором) получает в область сохранения ответ, содержащий вид выделенного ресурса и его адрес, активизируется и продолжает работу. В противном случае согласно запросу заполняется строка **ТС**, отведенная этому процессу. Обратившийся к операции запроса процесс блокируется, а запускается наиболее приоритетный готовый процесс.

TP(указатель на список семафоров ресурсов, указатель на список событий, тайм-аут)

begin

<задержать текущий процесс>;

for i:=1 to <число семафоров в запросе> do

if <счетчик i-го семафора >0> then

<взять ресурс из очереди; передать в область сохранения процесса адреса семафора и ресурса; активизировать задержанный процесс>;

end; end;

ТАБЛИЦА[ТЕКУЩИЙ, ГОТОВНОСТЬ]:= НЕ ГОТОВ;

ТАБЛИЦА[ТЕКУЩИЙ, ТАЙМЕР]:= ТАЙМ-АУТ;

ТАБЛИЦА[ТЕКУЩИЙ, РЕСУРСЫ]:= АДРЕС СПИСКА СЕМАФОРОВ;

<отметить в соответствующих столбцах таблицы заказанные в запросе события>;

<активизировать наиболее приоритетный готовый процесс>;

end;

Операцией **TV** процесс возвращает монитору освободившийся ресурс. Поиск процесса, ожидающего этот ресурс, осуществляется сканированием столбца **ТС РЕСУРСЫ** и просмотром списков ожидаемых каждым процессом ресурсов. Из всех ожидающих этот ресурс процессов находится самый приоритетный, в область сохранения которого передаются адреса семафора и ресурса. В зависимости от приоритета активизируется либо задержанный процесс, либо разблокированный. При отсутствии процессов, ожидающих этот ресурс, он возвращается в очередь к своему семафору и активизируется задержанный процесс.

TV(семафор, указатель на ресурс)

begin

<задержать текущий процесс>;

for i:=1 to <число строк в таблице> do

if ТАБЛИЦА[ТЕКУЩИЙ, РЕСУРСЫ]_{не}=0 then

if <список ожидаемых ресурсов содержит запрос на возвращаемый ресурс> then

<передать в область сохранения процесса P_i адрес семафора и ресурса>;

for j:=3 to <число столбцов в таблице> do

ТАБЛИЦА[i, j]:= 0;

end;

ТАБЛИЦА[i, ГОТОВНОСТЬ]:=ГОТОВ;

if i<ТЕКУЩИЙ then <активизировать P_i > else <активизировать задержанный процесс>

end; end; end; end;

<возвратить ресурс в очередь к семафору>;

<активизировать задержанный процесс>;

end;

Операцией **WAIT** процесс может заблокировать себя до наступления одного из нескольких событий. Можно задать тайм-аут ожидания событий. В столбцах **ТС** отмечаются перечисленные в запросе события.

WAIT(список событий, тайм-аут)

begin

<задержать текущий процесс>;

ТАБЛИЦА[ТЕКУЩИЙ, ГОТОВНОСТЬ]:= НЕ ГОТОВ;

ТАБЛИЦА[ТЕКУЩИЙ, ТАЙМЕР]:= ТАЙМ-АУТ;

<отметить в соответствующих столбцах таблицы заказанные в запросе события>;

<активизировать наиболее приоритетный готовый процесс>;

end;

Операцией **POST** процесс может сигнализировать о наступлении события. В результате наиболее приоритетный процесс из числа ожидающих данное событие переводится в состояние готовности, но активизируется только в том случае, если его приоритет выше приоритета процесса, выдавшего операцию **POST**.

POST(событие)

begin

<задержать текущий процесс>;

for i:=1 to <число строк в таблице> do

if ТАБЛИЦА[i, СОБЫТИЕ]не=0 then

for j:=3 to <число столбцов в таблице> do

ТАБЛИЦА[i, j]:=0;

end;

<переслать в область сохранения информацию о совершившемся событии>;

ТАБЛИЦА[i, ГОТОВНОСТЬ]:= ГОТОВ;

if i<ТЕКУЩИЙ then <активизировать P_i> else <активизировать задержанный процесс>

end; end; end;

<активизировать задержанный процесс>;

end;

Для обеспечения временной синхронизации необходим процесс **CLOCK**, имеющий бесконечный цикл работы и приоритет, равный приоритету процедур монитора. Процесс использует системный таймер вычислителя и запускает на один цикл работы таймера текущий процесс, после чего процесс **CLOCK** уменьшает на единичный интервал ненулевые элементы столбца ТАЙМЕР ТС, переводит процессы с истекшими тайм-аутами в состояние готовности, выбирает наиболее приоритетный готовый процесс и изменяет согласно выбору переменную ТЕКУЩИЙ.

Алгоритм процесса CLOCK

begin

do while (true) <запустить на 1 цикл работы таймера ТЕКУЩИЙ процесс>

<задержать текущий процесс>;

for i:=1 to <число строк в таблице> do

if ТАБЛИЦА[i, ТАЙМЕР]≠0 then

ТАБЛИЦА[i, ТАЙМЕР]:= ТАБЛИЦА[i, ТАЙМЕР]-1;

if ТАБЛИЦА[i, ТАЙМЕР]=0 then

for j:=3 to <число столбцов в таблице> do

ТАБЛИЦА[i, j]:=0;

end;

<переслать в область сохранения информацию о истекшем тайм-ауте>;

ТАБЛИЦА[i, ГОТОВНОСТЬ]:= ГОТОВ;

end; end; end; end;

Описанный монитор позволяет реализовать сложную схему меж процессного взаимодействия и решить задачу параллельного ожидания ресурсов и событий. Управление взаимодействием процессов с помощью ТС имеет ясную логическую структуру, является наглядным и простым (обусловлено концентрацией управляющей информации в ТС). Модификация меж процессных связей и появление новых процессов приведет к незначительным изменениям в ТС без изменения логики процедур.

ТУПИКИ

Ситуация, когда процессы ждут друг друга неопределенно много – ТУПИК!

Необходимые (но не достаточные) условия тупика:

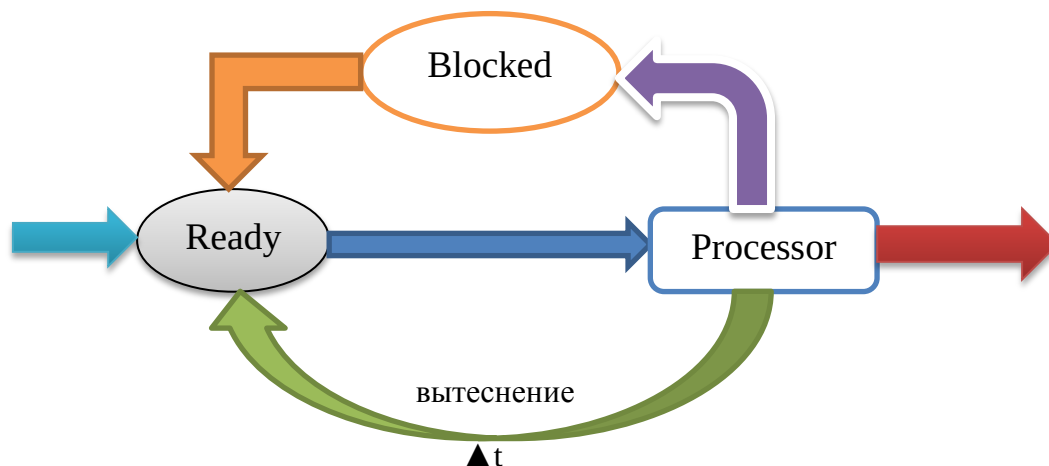
1. Условие **взаимоисключения** (процессы требуют монопольного управления выделенными им ресурсами).
2. Условие **удержания и ожидания** ресурсов (процессы удерживают за собой уже выделенные ресурсы и ожидают выделения дополнительных ресурсов).
3. Условие **неперераспределяемости** ресурсов (ресурсы нельзя отобрать у процессов, удерживающих их, пока они не будут использованы для завершения работы).
4. Условие **кругового ожидания** (существует кольцевая цепь процессов, в которой каждый процесс удерживает за собой один или более ресурсов, требующихся следующему процессу цепи).

ЛЕКЦИЯ 4. РАСПРЕДЕЛЕНИЕ ВРЕМЕНИ ПРОЦЕССОРА

Главная причина введения принципа мультипрограммирования – стремление компенсировать различие в скоростях ЦП и ВнУ.

Задание пользователя активно, если оно находится в ОЗУ и может выполняться. Совокупность активных заданий – рабочая смесь. Разделяют два режима мультипрограммирования. Состояния процесса: заблокированный, готов, выполняемый.

1. **Разделение времени** – обслуживание задач только в периоды выполнения операций В/В более приоритетными процессами==выполнение задач в фоновом режиме (такой режим планирования используется в системах жесткого РЕАЛЬНОГО времени).
2. **Квантование времени** 1+выделение кванта времени на обслуживание каждого из процессов рабочей смеси в соответствии с некоторой стратегией. Процессу выделяется квант времени и он выполняется до тех пор пока не произойдет одно из условий: а) истечет квант времени, б) процесс заблокирован операциями В/В, в) процесс завершится, г) процесс вытеснен более приоритетным процессом. При этом процесс либо попадает в очередь готовых процессов (а,г), либо попадает в очередь В/В (б), либо выходит из системы (в). Процесс переводится из очереди заблокированных процессов в очередь готовых процессов при снятии блокировки.



Алгоритм планирования очередей готовых процессов включает:

1. Способ выбора готового процесса из очереди готовых процессов **Ready**.
2. Способ вычисления размера кванта времени Δt для выбранного процесса.

Каждому процессу может ставиться в соответствие приоритет (число) означающее важность процесса. Вычисление приоритета основано на статических и динамических характеристиках процесса.

Статические характеристики:

1. Объем занимаемой памяти.
2. Ожидаемое время выполнения.
3. Ожидаемый объем В/В.
4. Коэффициент важности пользователя (внешний приоритет) – влияет на ввод задания в вычислительную систему и перевод в активное состояние.

Динамические характеристики:

1. Ресурсы, находящиеся в распоряжении процесса в данный момент.
2. Общее время ожидания.
3. Количество процессорного времени, полученного в течение предыдущего периода $T \gg \Delta t$ (окно наблюдения).
4. Объем операций В/В в течение предыдущего периода $T_{в/в} \gg T \gg \Delta t$ (окно наблюдения).
5. Общее время нахождения в системе.

При планировании очередей готовых процессов учитываются показатели производительности вычислительной системы:

1. Загрузка ресурсов вычислителя.
2. Времена простоев ресурсов вычислителя.
3. Пропускная способность.
4. Времена ожидания заданий пользователей.

Различают 3 основных класса алгоритмов планирования:

1. По наивысшему приоритету (HPF).
2. Круговорот.
3. Планирование с обратной связью.

ПЛАНИРОВАНИЕ ПО НАИВЫСШЕМУ ПРИОРИТЕТУ:

Если **вытеснение не допускается**, то процесс с наивысшим приоритетом выполняется до тех пор, пока не а) заблокируется, б) завершится. Если в очередь поступает более приоритетный процесс чем текущий, то он должен ждать пока текущий не освободит процессор.

Если **вытеснение разрешено**, то при появлении более приоритетного процесса, текущий процесс прерывается, направляется в очередь готовых процессов и управление передается вновь прибывшему.

ПЛАНИРОВАНИЕ ПО КРУГОВОРОТУ не использует информацию о приоритетах.

Первый процесс из очереди готовых получает квант времени Δt , а затем отправляется в конец очереди, если себя не заблокирует. При **К** процессах каждый процесс выполняется со скоростью, пропорциональной $1/K$. Квант времени при этом необходимо ограничивать снизу временем переключения процессора t ($\Delta t \gg t$).

ПЛАНИРОВАНИЕ С ОБРАТНОЙ СВЯЗЬЮ использует набор из N очередей готовых процессов, каждая из которых обслуживается в порядке поступления (FCFS).

Новый процесс в системе попадает в 1 очередь.

После получения первого кванта и обслуживания на процессоре поступает во 2 очередь.

После получения каждого последующего кванта и обслуживания на процессоре поступает в очередь со следующим номером.

В очереди N процесс обслуживается до завершения.

Кванты времени с ростом номера очереди увеличиваются $\Delta t(i) < \Delta t(i+1)$, $i=1, \dots, N-1$.

Время процессора планируется так, чтобы он **обслуживал непустую очередь с наименьшим номером**.

Планирование с обратной связью обеспечивает лучшее, чем при круговороте обслуживание коротких процессов и не требует никакой предварительной информации (трудоемкие процессы быстро проваливаются в последнюю очередь и не мешают выполнению коротких).

Практика применения алгоритмов планирования.

1. ОС **Реального времени** и **Встроенные системы** используют планирование по наивысшему приоритету.
2. ОС **Разделения времени** и **Диалоговые системы** используют планирование по наивысшему приоритету и круговорот.
3. ОС **Пакетной обработки** используют планирование с обратной связью.

МНОГОУРОВНЕВОЕ ПЛАНИРОВАНИЕ

При реализации планирования необходимо выполнение набора работ:

1. Работа с очередями.
2. Переключение процессов.
3. Вычисление приоритетов.
4. Текущие измерения характеристик процессов.

Метод многоуровневого планирования подразумевает несколько уровней:

1. Диспетчер
2. Краткосрочный планировщик.
3. Долгосрочный планировщик.

Цель: минимизировать затраты на переключение процессов (часто встречающиеся операции должны быть оптимизированы и иметь низкую трудоемкость, в отличие от редких операций).

Диспетчер – вызывается после обработки прерывания, выбирает процесс из очереди готовых процессов и запускает его.

Краткосрочный планировщик – вызывается, чтобы поставить готовый процесс в очередь готовых процессов (выстраивает очередь готовых процессов для диспетчера).

Долгосрочный планировщик – обработка информации о состоянии процесса, предоставленных ресурсах, вычисление приоритета и размера кванта (готовит управляющую информацию для краткосрочного планировщика).

При такой 3-х уровневой организации издержки времени, связанные с планированием, локализуются на различных уровнях.

УПРАВЛЕНИЕ ПАМЯТЬЮ

Управление памятью – отображение информации в память с помощью 3-х функций:

1. **Именуемая функция (И)** – однозначно отображает данное пользователем имя (**ПИ**) в идентификатор информации (программный адрес – **ПА**), к которой это имя относится (переменные, файлы и др. объекты).
2. **Функция памяти (П)** – отображает идентификаторы (программные адреса) в номера физических ячеек памяти (физические адреса – **ФА**), в которых они находятся.
3. **Функция содержимого/значения (З)** – отображает каждый адрес памяти в значение, которое находится по этому адресу.

Исходная информация для загрузчика, объединяющего модули (совокупность модулей + дополнительные данные):

1. Объектный модуль.
2. Длина модуля.
3. Имена описанные в модуле, на которые могут ссылаться другие модули (ссылки внутрь).
4. Имена не описанные в данном модуле, на которые есть ссылки в данном модуле (ссылки наружу).

Загрузчик, объединяющий модули, реализует объединение объектных модулей, настройку адресов, загрузку объединенной программы в ОЗУ.

Загрузчик составляет таблицу имен, на которые могут быть ссылки из других модулей (ссылки внутрь). В таблице каждому имени ставится в соответствие запись следующего содержания:

- а) Если имя описано в модуле, который уже загружен, то запись содержит адрес этого имени относительно начала объединенного имени.
- б) Если объектный модуль, в котором определено это имя, еще не загружен (адрес неизвестен), то запись содержит ссылку на связанный список адресных полей, в которых в дальнейшем нужно будет поместить адрес, соответствующий данному имени. Когда это имя встретится загрузчику, он просмотрит список и вставит адрес этого имени по всем указателям списка.

Все модули загружаются последовательно в ОЗУ. Относительные адреса сдвигаются на сумму длин уже загруженных модулей. Каждая внешняя ссылка либо разрешается (если модуль уже загружен), либо присоединяется к списку, соответствующему этому имени. После загрузки таблицу можно сохранить для отладки.

Узловые моменты привязки приложения к памяти:

1. Программирование в абсолютных адресах – при написании программы указываются абсолютные адреса (объединение отображений **И** и **П** раз и навсегда).
2. Программирование в символьных обозначениях – компилятор порождает адреса вместо символьных имен.

А) Порождаемые адреса – фиксированные абсолютные адреса (привязка **П** происходит сразу же за **И**).

Б) Порождаемые адреса – допускают настройку и при запуске получают абсолютные значения, не меняющиеся при выполнении программы (привязка к памяти **П** происходит во время загрузки).

В) Порождаемые адреса допускают настройку и получают абсолютные значения при каждом обращении к ним (привязки **П** как таковой нет = динамическое распределение).

Основной вопрос задачи распределения памяти состоит в том как объединенный модуль записать в ОЗУ. Далее исходим из того. Что задания переместимы.

Стратегии распределения памяти.

1. Перекрытие программ в памяти (планируется пользователем).
2. Попеременная загрузка заданий.
3. Сегментная организация программ.

4. Страничная организация памяти.
5. Сочетание сегментной организации программ со страничной организацией памяти.

Перекрывание программ

Условие перекрывания: 2 модуля могут располагаться в одних и тех же ячейках памяти, если: а) модулям не нужно присутствовать в ОЗУ одновременно, б) для сохранения межмодульной коммуникационной информации (обычно через поле общей памяти) принимаются необходимые предосторожности.

Для реализации перекрывания редактор связей порождает специальные команды, вызывающие загрузчик всякий раз, когда происходит обращение к одной из перекрывающихся программ (загрузчик вводит модуль в ОЗУ, если он еще не загружен).

Проблемы:

Программист не всегда может априори задать описание перекрываний (т.к. может не знать нужно ли присутствовать перекрывающимся модулям в памяти одновременно).

Т.к. перекрывания планируются пользователем заранее, а степень загрузки памяти и поведение других программ предвидеть заранее невозможно, то эта стратегия не дает существенного улучшения использования всей памяти.

Попеременная загрузка заданий

ОС осуществляет загрузку/выгрузку заданий целиком после начальной загрузки в процессе выполнения смеси заданий. Стратегия позволяет избирательно загружать задания с целью увеличения загрузки все компонент вычислителя.

Сегментная организация программ

Сегментация программ – прием, при котором адресная структура программы отражает ее содержательное членение. Пространство адресов программы разделяется на отрезки – сегменты различной длины, соответствующие содержательно различным частям программы. Сегмент = процедура, область описания данных. При сегментации АДРЕС состоит из 2-х частей:

1. Имя сегмента **S** (его номер).
 2. Адрес слова внутри сегмента **d** (смещение).
- ⊕ а) Сегменты могут размещаться в несмежных областях памяти.
б) Не все сегменты должны одновременно находиться в памяти (часть может располагаться на ВнУ).

Реализация

Каждому заданию (приложению), состоящему из набора сегментов, соответствует ТАБЛИЦА СЕГМЕНТОВ (ТС), в которой для каждого сегмента имеется запись (строка). Адрес ТС хранится в аппаратном РЕГИСТРЕ ТАБЛИЦЫ СЕГМЕНТОВ (РТС) каждого процессора.

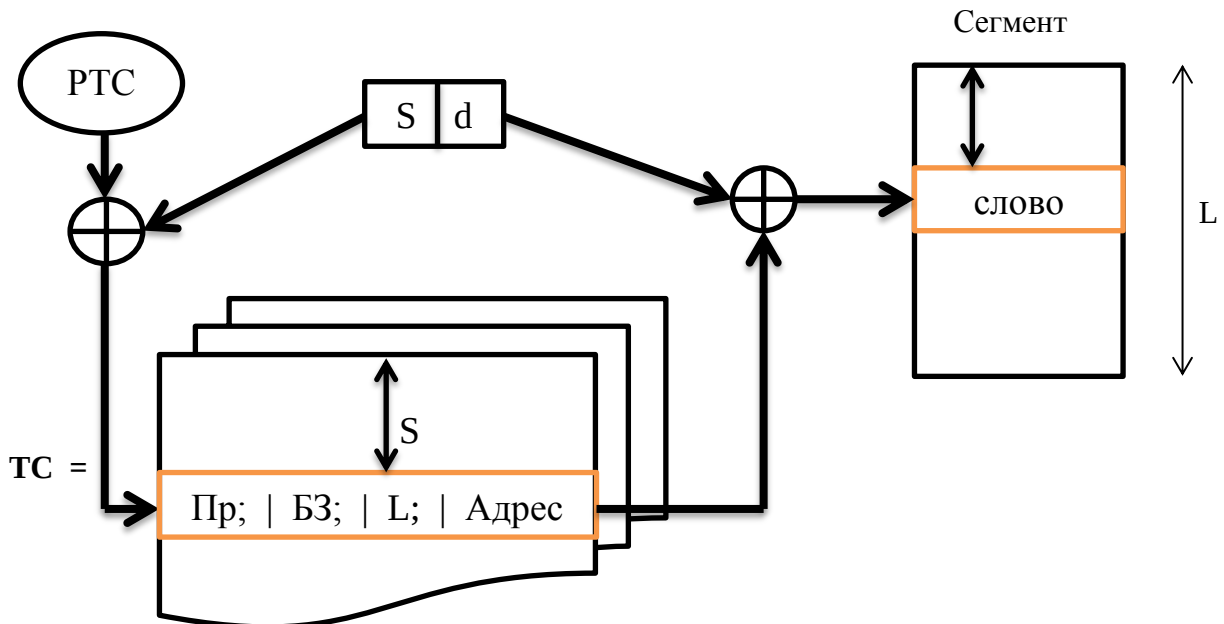
В S-й строке ТС (имя сегмента=№строки) находится:

1. Признак присутствия сегмента в ОЗУ (**Пр**).
2. Адрес (Базовый регистр) S-го сегмента (**Адрес**).
3. Длина (Граница) сегмента (**L**).
4. Биты защиты для контроля доступа к сегменту (**БЗ**).

Доступ к слову [S,d]:

1. По РТС обращение к ТС.
2. Проверяется присутствие сегмента в памяти (по признаку **Пр**)
3. При наличии сегмента в ОЗУ в S-й строке выбирается **Адрес** сегмента.
4. В d-й ячейке выбирается искомое слово.

Если сегмента в ОЗУ нет – происходит прерывание из-за отсутствия сегмента в памяти.



Переключение системы на другую программу (приложение) сводится к замене содержимого РТС на адрес другой таблицы.

- ⊕
- а) Сегменты могут размещаться в несмежных областях памяти.
 - б) Не все сегменты должны одновременно находиться в памяти (часть может располагаться на ВнУ).
 - в) Перекрытия реализуются системой без предварительных указаний программиста. Возможно перекрытие областей отдельных программ. При этом эффективно используется память.
 - г) Упрощается объединение модулей. Операция установления внешних связей сводится к просмотру таблицы для отыскания адреса (S,d), соответствующего внешней ссылке (не нужны эти функции в редакторе и загрузчике).
 - д) сегментация облегчает организацию разделения параллельно используемых программ. Для этого информацию о параллельно используемой программе вносят несколько таблиц сегментов. Сама программа при этом обрабатывает разные сегменты данных, описанные в таблицах различных приложений.

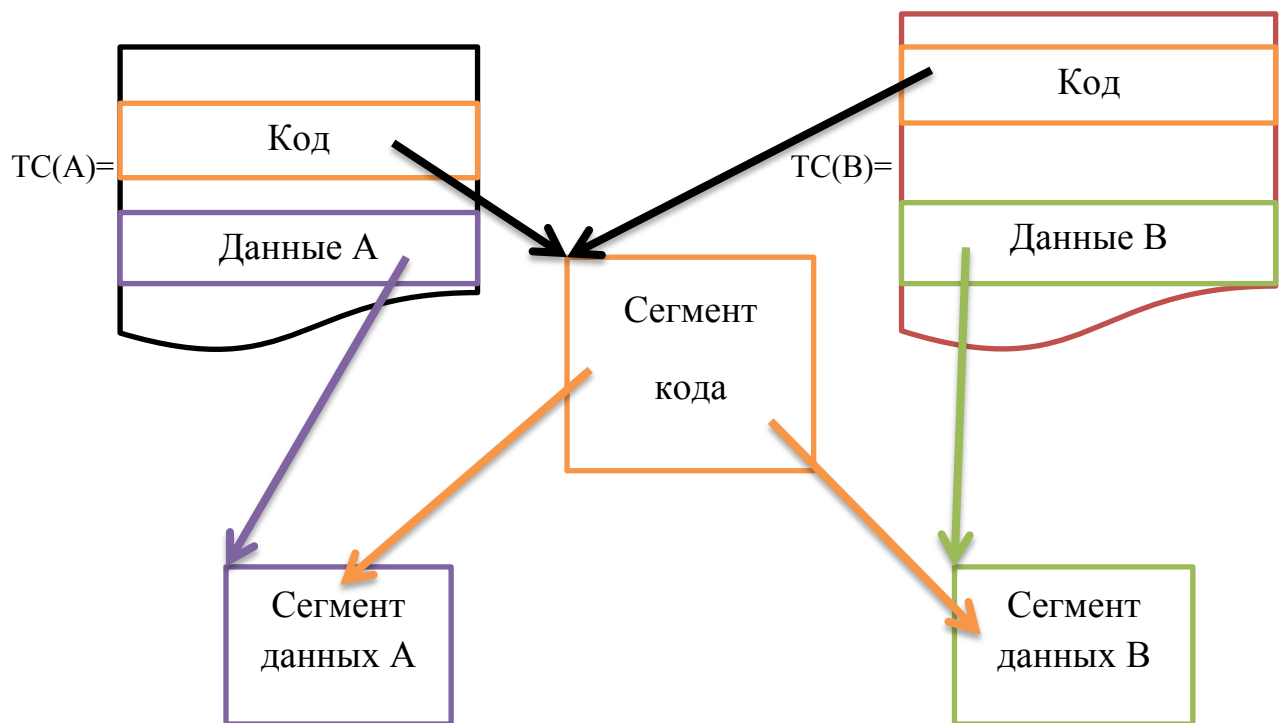
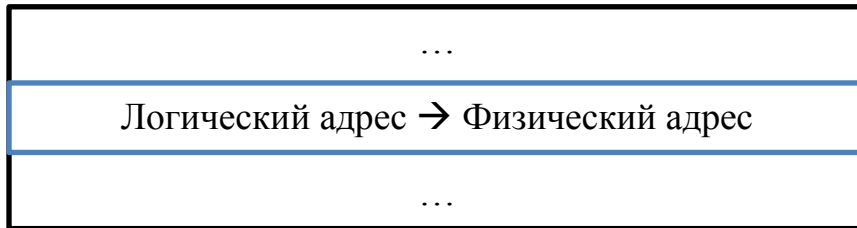


А) Внешняя фрагментация (пустоты между загруженными в память сегментами, размеры которых малы для размещения других сегментов – образуются из-за загрузки-выгрузки сегментов в процессе вычислений). Профилактика: страничная организация памяти!

Б) Два обращения к ОЗУ (первое – к таблице за адресом сегмента, второе – к адресуемому объекту внутри сегмента). Существенно снижает быстродействие вычислителя. Профилактика: полностью ассоциативное кэширование адресных ссылок на

сегменты (емкость кэша адресных ссылок TLB обычно несколько сотен строк), реализуется с помощью отображения:

Вход=ID-задачи+S (логический адрес)→ **Физический адрес сегмента**=**Выход**



УПРАВЛЕНИЕ ПАМЯТЬЮ

Управление памятью – отображение информации в память с помощью 3-х функций:

1. **Именуемая функция (И)** – однозначно отображает данное пользователем имя (**ПИ**) в идентификатор информации (программный адрес – **ПА**), к которой это имя относится (переменные, файлы и др. объекты).
2. **Функция памяти (П)** – отображает идентификаторы (программные адреса) в номера физических ячеек памяти (физические адреса – **ФА**), в которых они находятся.
3. **Функция содержимого/значения (З)** – отображает каждый адрес памяти в значение, которое находится по этому адресу.



Результат каждого отображения зависит от времени:

Именуемая – не устанавливается до тех пор, пока задание не будет связано с системными модулями и файлами, которые используются в задании.

Памяти – фиксируется после загрузки задания (как правило).

Значения – меняется всякий раз при выполнении команды записи в память. Эта функция полностью определяется алгоритмом программы (пользователем).

Отображения Именуемой функции и функции Памяти находятся в ведении системы.

При именовании данное пользователем имя может относиться к различным объектам в зависимости от контекста, в котором оно употребляется. Для ОС контекст это текст программы пользователя. Пользователи могут дать одни и те же имена различным объектам. ОС должна установить **соответствие каждому имени** в каждом контексте пользователя **нужного объекта**. Необходимо также, чтобы один объект мог именоваться разными именами в различных программах.

Структура программ и распределение памяти связаны и зависят друг от друга!

ОБЪЕДИНЕНИЕ МОДУЛЕЙ

Обработка программ включает этапы:

1. **Компиляция.** На вход подается *текст программы*, на выходе получаем *объектный код*.
2. **Сборка.** На вход подается *объектный код*, на выходе получаем *загрузочный код*.
3. **Загрузка.** На вход подается *загрузочный код*, на выходе получаем *рабочий код*.

Основная проблема этапов 2 и 3 – объединение модулей (разрешение внешних ссылок).

Процесс разрешения ссылок может быть выполнен 3-мя способами:

1. Объединение различных процедур во время компиляции. Реализуется в большинстве языков программирования.
2. Предварительно скомпилированные программы объединяются либо до (на этапе сборки) либо во время загрузки. Отдельный шаг между компиляцией и загрузкой.
3. Объединение откладывается до запуска на счет и осуществляется системой автоматически в процессе счета = динамическое разрешение ссылок. Данный способ

используется, когда имена некоторых процедур неизвестны до начала выполнения программы и характер вычислений зависит от входных данных.

СПОСОБЫ ОБЪЕДИНЕНИЯ МОДУЛЕЙ

Сборка как предварительный этап. Группа объектных модулей объединяется в 1 модуль в относительных адресах с началом в нулевой ячейке. Объединенный модуль – переместимый объект и при загрузке в ОЗУ абсолютный адрес первого слова помещается в **Базовый регистр**. Тогда абсолютный адрес это сумма содержимого базового регистра и относительного адреса в объединенном модуле. После редактирования задание – переместимый модуль загрузки, начинающийся с нулевого адреса.

Сборка при загрузке. Исходная информация для загрузчика, объединяющего модули (совокупность модулей + дополнительные данные):

1. Объектный модуль.
2. Длина модуля.
3. Имена описанные в модуле, на которые могут ссылаться другие модули (ссылки внутрь).
4. Имена не описанные в данном модуле, на которые есть ссылки в данном модуле (ссылки наружу).

Загрузчик, объединяющий модули, реализует объединение объектных модулей, настройку адресов, загрузку объединенной программы в ОЗУ.

Загрузчик составляет таблицу имен, на которые могут быть ссылки из других модулей (ссылки внутрь). В таблице каждому имени ставится в соответствие запись следующего содержания:

- а) Если имя описано в модуле, который уже загружен, то запись содержит адрес этого имени относительно начала объединенного имени.
- б) Если объектный модуль, в котором определено это имя, еще не загружен (адрес неизвестен), то запись содержит ссылку на связанный список адресных полей, в которых в дальнейшем нужно будет поместить адрес, соответствующий данному имени. Когда это имя встретится загрузчику, он просмотрит список и вставит адрес этого имени по всем указателям списка.

Все модули загружаются последовательно в ОЗУ. Относительные адреса сдвигаются на сумму длин уже загруженных модулей. Каждая внешняя ссылка либо разрешается (если модуль уже загружен), либо присоединяется к списку, соответствующему этому имени. После загрузки таблицу можно сохранить для отладки.

Узловые моменты привязки приложения к памяти:

1. Программирование в абсолютных адресах – при написании программы указываются абсолютные адреса (объединение отображений **И** и **П** раз и навсегда).
2. Программирование в символьных обозначениях – компилятор порождает адреса вместо символьных имен.

А) Порождаемые адреса – фиксированные абсолютные адреса (привязка **П** происходит сразу же за **И**).

Б) Порождаемые адреса – допускают настройку и при запуске получают абсолютные значения, не меняющиеся при выполнении программы (привязка к памяти **П** происходит во время загрузки).

В) Порождаемые адреса допускают настройку и получают абсолютные значения при каждом обращении к ним (привязки **П** как таковой нет = динамическое распределение).

Основной вопрос задачи распределения памяти состоит в том, как объединенный модуль записать в ОЗУ. Далее исходим из того, что задания переместимы.

Стратегии распределения памяти.

1. Перекрывание программ в памяти (планируется пользователем).
2. Попеременная загрузка заданий.
3. Сегментная организация программ.
4. Страничная организация памяти.
5. Сочетание сегментной организации программ со страничной организацией памяти.

ПЕРЕКРЫТИЕ ПРОГРАММ

Условие перекрывания: 2 модуля могут располагаться в одних и тех же ячейках памяти, если: а) модулям не нужно присутствовать в ОЗУ одновременно, б) для сохранения межмодульной коммуникационной информации (обычно через поле общей памяти) принимаются необходимые предосторожности.

Для реализации перекрывания редактор связей порождает специальные команды, вызывающие загрузчик всякий раз, когда происходит обращение к одной из перекрывающихся программ (загрузчик вводит модуль в ОЗУ, если он еще не загружен).

Проблемы:

Программист не всегда может априори задать описание перекрываний (т.к. может не знать нужно ли присутствовать перекрывающимся модулям в памяти одновременно).

Т.к. перекрывания планируются пользователем заранее, а степень загрузки памяти и поведение других программ предвидеть заранее невозможно, то эта стратегия не дает существенного улучшения использования всей памяти (особенно при низкой нагрузке на вычислитель).

ПОПЕРЕМЕННАЯ ЗАГРУЗКА ЗАДАНИЙ

ОС осуществляет загрузку/выгрузку заданий целиком после начальной загрузки в процессе выполнения смеси заданий. Стратегия позволяет избирательно загружать задания с целью увеличения загрузки всех компонент вычислителя. Основной недостаток – внешняя фрагментация (пустоты памяти между заданиями недостаточного размера для загрузки других заданий).

СЕГМЕНТНАЯ ОРГАНИЗАЦИЯ ПРОГРАММ

Сегментация программ – прием, при котором адресная структура программы отражает ее содержательное членение. Пространство адресов программы разделяется на отрезки – сегменты различной длины, соответствующие содержательно различным частям программы. Сегмент = процедура, область описания данных. При сегментации АДРЕС состоит из 2-х частей:

1. Имя сегмента **S** (его номер).
2. Адрес слова внутри сегмента **d** (смещение).

Реализация

Каждому заданию (приложению), состоящему из набора сегментов, соответствует ТАБЛИЦА СЕГМЕНТОВ (ТС), в которой для каждого сегмента имеется запись (строка). Адрес ТС хранится в аппаратном РЕГИСТРЕ ТАБЛИЦЫ СЕГМЕНТОВ (РТС) каждого процессора.

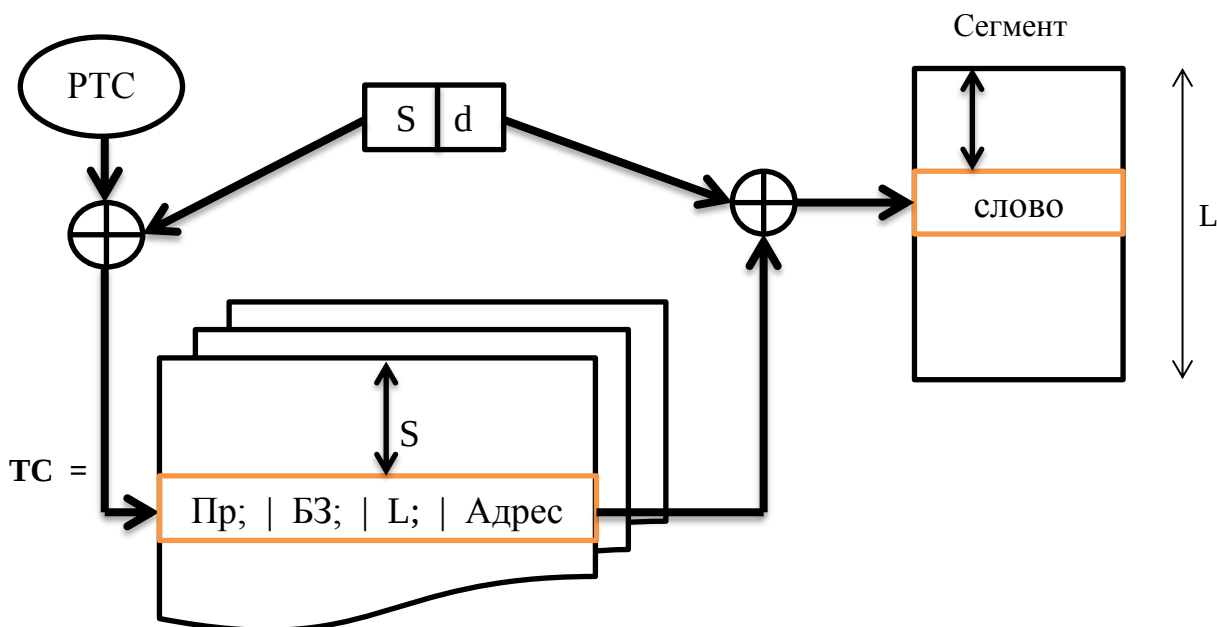
В S-й строке ТС (имя сегмента=№строки) находится:

1. Признак присутствия сегмента в ОЗУ (**Пр**).
2. Адрес (Базовый регистр) S-го сегмента (**Адрес**).
3. Длина (Граница) сегмента (**L**).
4. Биты защиты для контроля доступа к сегменту (**БЗ**).

Доступ к слову [S,d]:

1. По РТС обращение к ТС.
2. Проверяется присутствие сегмента в памяти (по признаку **Пр**)
3. При наличии сегмента в ОЗУ в S-й строке выбирается **Адрес** сегмента.
4. В d-й ячейке выбирается искомое слово.

Если сегмента в ОЗУ нет – происходит прерывание из-за отсутствия сегмента в памяти.



Переключение системы на другую программу (приложение) сводится к замене содержимого РТС на адрес другой таблицы.

- ⊕
- а) Сегменты могут размещаться в несмежных областях памяти.
 - б) Не все сегменты должны одновременно находиться в памяти (часть может располагаться на ВнУ).
 - в) Перекрытия реализуются системой без предварительных указаний программиста. Возможно перекрытие областей отдельных программ. При этом эффективно используется память.
 - г) Упрощается объединение модулей. Операция установления внешних связей сводится к просмотру таблицы для отыскания адреса (S,d), соответствующего внешней ссылке (не нужны эти функции в редакторе и загрузчике).
 - д) сегментация облегчает организацию разделения параллельно используемых программ. Для этого информацию о параллельно используемой программе вносят несколько таблиц сегментов. Сама программа при этом обрабатывает разные сегменты данных, описанные в таблицах различных приложений.

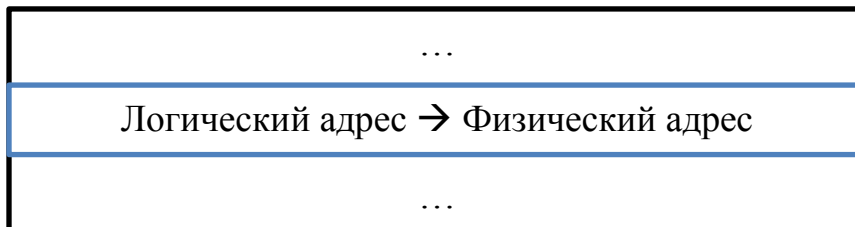


А) Внешняя фрагментация (пустоты между загруженными в память сегментами, размеры которых малы для размещения других сегментов – образуются из-за загрузки-выгрузки сегментов в процессе вычислений). Профилактика: страничная организация памяти!

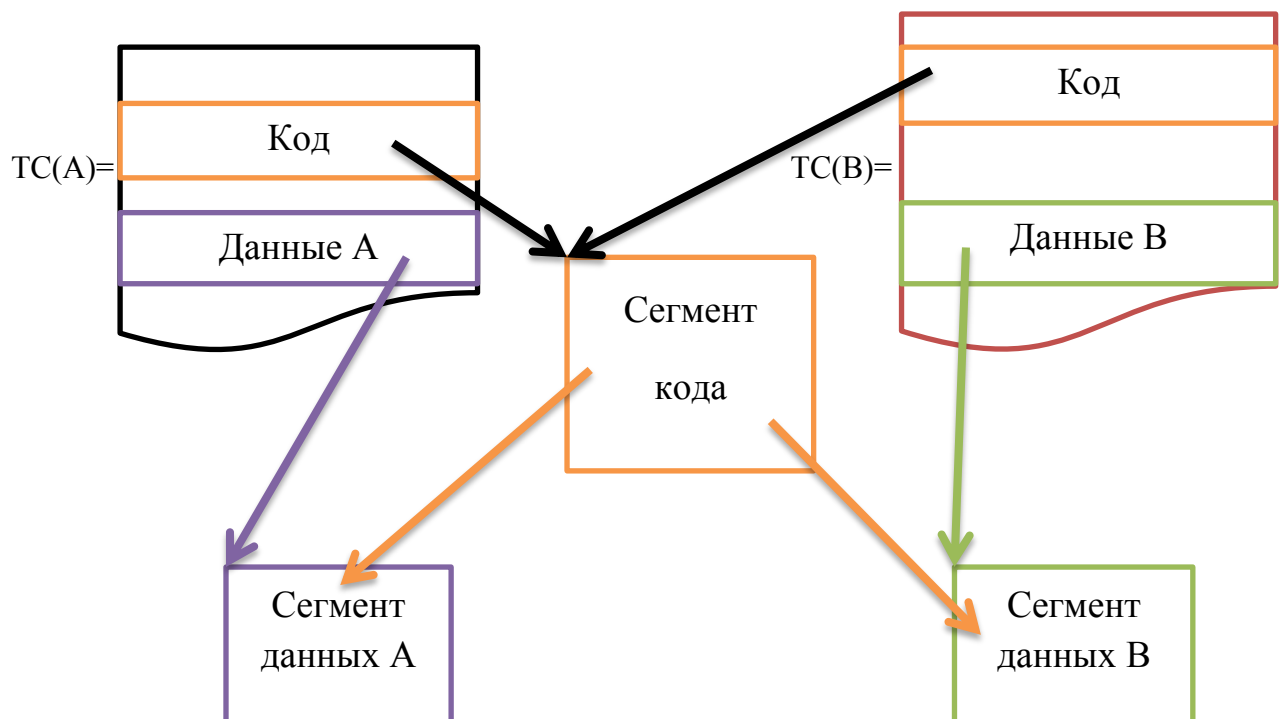
Б) Высокие накладные расходы на сопровождение миграции сегментов между ОЗУ и внешней памятью. Профилактика: страничная организация памяти!

В) Два обращения к ОЗУ (первое – к таблице за адресом сегмента, второе – к адресуемому объекту внутри сегмента). Существенно снижает быстродействие вычислителя. Профилактика: полностью ассоциативное кэширование адресных ссылок на сегменты (емкость кэша адресных ссылок TLB обычно несколько сотен строк), реализуется с помощью отображения:

Вход=ID-задачи+S (логический адрес)→ Физический адрес сегмента=Выход



В качестве ID-задачи может выступать АДРЕС ТС (значение РТС).



СТРАНИЧНАЯ ОРГАНИЗАЦИЯ ПАМЯТИ

Страничная организация памяти – способ управления памятью, при котором пространство адресов памяти разбивается на блоки фиксированной длины – страницы (обычно размер = степень двойки).

Память – набор страниц равной длины.

Программы и данные – делятся на страницы того же размера, что и физические страницы ОЗУ, при этом программа/данные занимает целое число страниц, последняя страница может быть заполнена не полностью. Страница информации загружается в физическую страницу памяти.

Адрес: пара $[P, d]$, P – имя (номер) страницы, d – смещение. Задание – таблица страниц (ТСтр) содержит список страниц, составляющих это задание. Запись в таблице соответствует странице и содержит:

1. Признак наличия страницы в памяти.
2. Указатель местоположения страницы (Адрес страницы).
3. Биты защиты для контроля способа доступа.

Регистр таблицы страниц (РТСтр) центрального процессора указывает местоположение таблицы страниц текущего (выполняемого в данный момент) задания.

Доступ к объекту по логическому адресу $[P, d]$:

1. По регистру РТСтр выполняется обращение к ТСтр.
2. В P -й записи (строке) ТСтр выбирается адрес страницы.
3. В d -й ячейке страницы находим искомое слово.

Если требуемой страницы нет в ОЗУ, то происходит прерывание из-за отсутствия страницы.

 **Плюсы** – те же, что и при сегментной организации программ, **кроме** возможности организации совместного параллельного использования программ. **Дополнительно:** в отличие от сегментной организации программ при страничной организации памяти существенно легче реализуется миграция страниц на различные уровни подсистемы памяти.



А) Два обращения к ОЗУ. Профилактика – кэширование отображения логических адресов страниц в физические (решается, так же как и в случае сегментной организации программ).

Б) Внутренняя фрагментация. Пустоты возникают внутри последней страницы приложения. Возникает задача поиска компромисса между: издержками, связанными с движением страниц (желательно размер страницы увеличивать), и издержками внутренней фрагментации (желательно размер страницы уменьшать).

СОЧЕТАНИЕ СЕГМЕНТНОЙ ОРГАНИЗАЦИИ ПРОГРАММ И СТРАНИЧНОЙ ОРГАНИЗАЦИИ ПАМЯТИ

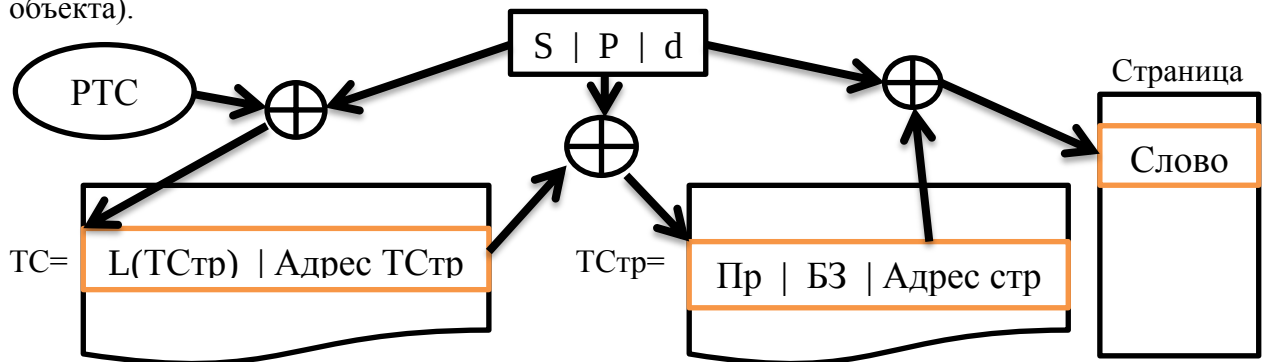
Память вычислителя разбивается на страницы фиксированной длины.

Программа – набор сегментов.

Сегмент – набор страниц.

Адрес – тройка: $[S, P, d]$, S – номер сегмента приложения, P – номер станицы данного сегмента S , d – смещение внутри страницы P .

Для выбора адресуемого объекта при такой трехкомпонентной ссылке нужно 3 цикла обращения к памяти (в таблицу сегментов за адресом таблицы страниц, в таблицу страниц за адресом страницы с адресуемым объектом, в страницу за значением адресуемого объекта).



$\oplus$ Сегментно-страничному способу распределения памяти присущи все преимущества сегментной организации программ и страничной организации памяти (сегментация позволяет просто организовать совместное использование кода программ и установление внешних связей, страничная организация обеспечивает эффективное распределение памяти и исключает внешнюю фрагментацию).



Такая высокая способность приспосабливаться к различным требованиям достигается ценой 3-х компонентной адресной ссылки (3-х обращений к ОЗУ). Профилактика – кэширование отображения:

Вход=ID-задачи+S+P (лог. адрес) $\rightarrow$ **Физический адрес страницы**=**Выход**

ВИРТУАЛЬНАЯ ПАМЯТЬ

В системе с ВП у каждого процесса создается иллюзия. Что все приложение находится в ОЗУ. При обращении процесса к информации, которой нет в ОЗУ, вырабатывается прерывание, управление передается супервизору, который перемещает нужную информацию с внешнего устройства в ОЗУ и процесс получает доступ к ней.

В самом общем случае ВП организуется в виде многоуровневой памяти. Каждый уровень $M(i)$ имеет более высокую скорость доступа, меньшую емкость и более высокую удельную стоимость хранения по сравнению с уровнем $M(j)$ при $i < j$. Как правило, многоуровневая память разбита на страницы фиксированного размера. Объект, который находится в памяти i -го уровня, хранится также на уровнях с большими номерами. Если ЦП обращается к объекту, которого нет в памяти 1-го уровня, система ищет его на уровнях с большими номерами. Когда объект найден, он перемещается на верхние уровни вплоть до 1-го. При переносе объекта с более низкого уровня на более высокий (более быстрый) вытесняется какой либо объект – согласно стратегии вытеснения.

СТРАТЕГИИ РАСПРЕДЕЛЕНИЯ РЕСУРСОВ ПРИ СЕГМЕНТАЦИИ И СТРАНИЧНОЙ ОРГАНИЗАЦИИ.

Сегментация и страничная организация – это логический и физический подходы применяемые к пространству программных адресов и пространству адресов памяти.

Существенное различие между разбиением на сегменты и страницы состоит в том, что сегменты – блоки переменной длины, страницы – блоки фиксированной длины. Отсюда проблемы: размещения логических сегментов в памяти, разбиение программ на страницы.

Важнейшие вопросы реализации:

1. Как распределять память каждого уровня (для страничной организации задача проста – ОС ведет таблицу программных страниц и координат соответствующих физических страниц, для сегментации задача более сложная).
2. Какие критерии применять при подкачке блоков (различают 2 стратегии: опережающая и по запросу).
3. Какие критерии применять при вытеснении блоков (существует много стратегий).

1. СТРАТЕГИИ РАСПРЕДЕЛЕНИЯ ДЛЯ СЕГМЕНТОВ ПЕРЕМЕННОЙ ДЛИНЫ

Задача: найти стратегию, которая хорошо загружает память при минимальных издержках распределяющих алгоритмов.

Решение: список свободной памяти или уплотнение.

ВАРИАНТЫ СПИСКОВ СВОБОДНОЙ ПАМЯТИ.

Список свободной памяти, упорядоченный по возрастанию адресов.

Каждая пустота, входящая в список, содержит указатель на следующую и поле длины (количество блоков пустой области). Освобожденная область вставляется в список с сохранением упорядоченности адресов по возрастанию. Если предшествующая и (или)

последующая область списка пустот примыкают к вновь освобожденной, то они объединяются в одну пустоту.

Из списка пустот память может распределяться по 2-м стратегиям:

1. Выбор 1-й подходящей пустоты (при запросе пустоты размера k выделяется первая пустота размера $\geq k$).
2. Выбор самой подходящей пустоты (просматривается весь список и выбирается наименьшая пустота размера $\geq k$).

Если нет пустоты размера $\geq k$, то запрос остается неудовлетворенным, либо уплотнение (если сумма длин пустот в списке $\geq k$).

Во многих случаях 1-я стратегия лучше заполняет память, чем 2-я.

Список свободной памяти, упорядоченный по возрастанию размера пустот.

Поиск первой подходящей пустоты дает самую подходящую.

Модификация:

Список пустот дополняется списком указателей $\{P_1, P_2, \dots, P_m\}$, P_i – указывает на первую пустоту в исходном списке размера от iC до $(i+1)C$ блоков ($C = \text{const}$). Этот набор указателей позволяет ускорить поиск пустоты нужного размера, но при высокой динамике содержания списка его сопровождение очень дорогое. Главная трудность состоит в сложности вставки вновь освободившейся области в список пустот (требует много времени).

Организация пустот системой расщепления

Память вычислителя представляется блоками длины 2^k , $k \geq 0$.

Список k содержит блоки 2^k .

Пусть объем распределяемой памяти $= 2^s$, тогда в системе всего $S+1$ список $0 \leq k \leq s$. Сначала все списки пусты, кроме Списка S , который указывает на первое слово в распределяемой памяти.

При поступлении запроса на пустоту размера 2^k :

1. Просматривается Список k , если он не пуст, то выделяется пустота иначе...
2. Просматривается Список $k+1$, если он не пуст, один из его блоков расщепляется на 2 половинки, одна выделяется по запросу, а другая помещается в Список k , иначе...
3. Процесс поиска пустоты продолжается до тех пор, пока не будет найден непустой Список.
4. Если Списки J для $k \leq J \leq s$, то запрос не удовлетворяется.

При таком способе организации управления списками адрес каждого блока длины 2^k без остатка делится на 2^k , тогда для каждого блока размера 2^k легко найти адрес того блока длины 2^{k+1} , от которого он был отщеплен. Как только два парных блока длины 2^k – двойняшки, получившиеся в результате расщепления одного блока длины 2^{k+1} окажутся свободными, они объединяются.

УПЛОТНЕНИЕ

Если запрошена область длины превосходящей наибольшую пустоту, то возможны следующие действия:

1. Отказ в удовлетворении запроса.
2. Вытеснение какого-либо сегмента.
3. Уплотнение – перегруппировка памяти и объединение мелких пустот.

Обычно уплотнение выгодно когда: запросы на память велики по размеру и из-за внешней фрагментации образуется большой процент пустот, ЦП не занят выполнением активных заданий (простаивает) и его можно занять уплотнением – операция будет бесплатной.

2. СТРАТЕГИИ ПОДКАЧКИ ОБЪЕКТОВ

2 стратегии: **по запросу** (по прерыванию), **опережающая**.

Экономия опережающей подкачки определяется исключением прерываний (их число определяется объемом опережения) = затраты на обслуживание прерываний – затраты на опережающий выбор объектов. Этот выигрыш имеет место при условии. Что объекты будут введены не преждевременно и к ним будет обращение до вытеснения (при высокой вероятности использования объектов)! Обычно это наблюдается при известной логической связи между объектами (например – медиа файл). В большинстве случаев подкачка по запросу выгоднее опережающей подкачки.

3. СТРАТЕГИИ ВЫТЕСНЕНИЯ ОБЪЕКТОВ ИЗ ПАМЯТИ

Правила вытеснения оцениваются и сопоставляются по критерию того насколько хорошо они сокращают ожидаемый объем движения объектов между уровнями памяти. Нужно не вытеснять **НУЖНЫЕ** объекты!

Стратегии вытеснения:

1. **Случайный выбор** – легко реализуемая стратегия, но часто вытесняются **НУЖНЫЕ** объекты.
2. **FIFO** – первым пришёл – первым ушёл. Легко реализуется для страниц фиксированной длины – физические страницы образуют список из K элементов, упорядоченный по номерам страниц, указатель ссылается на самую новую страницу. Когда необходимо ввести какую-либо страницу содержимое указателя складывается по модулю K с единицей и страница, на которую указывает полученное значение, удаляется. Однако данная стратегия существенно ограничивает производительность системы т.к. самая старая страница не обязательно наименее нужная.
3. **По давности использования (LRU – Least Recently Used)**. Удаляется тот объект, обращения к которому не было дольше, чем к все остальные объекты. Трудно реализуемая дорогая стратегия (требует мониторинга частоты обращения), но в большинстве случаев работает хорошо!
4. **Предписанные пользователем (программистом) приоритеты**. Используется при известной связи приоритетов с **НУЖНОСТЬЮ** объектов.
5. **Приоритеты, задаваемые системой**.
6. **Оптимальное вытеснение**. Вытесняется тот объект, обращение к которому не произойдет дольше всего. Стратегия не реализуема т.к. в любой момент времени нельзя знать о будущих обращениях к памяти, **НО** используется как эталон для реализуемых стратегий при апостериорном анализе свойств реализуемой стратегии.
7. **Смешанные методы** – основаны на здравом смысле. Для вытеснения предлагается на выбор список возможностей, упорядоченных по убыванию степени их желательности. Например:
 - а) принадлежащий неактивной задаче объект, использованный только для чтения;
 - б) принадлежащий неактивной задаче объект, использованный только для записи;
 - в) объект управляющей программы, к которому не было обращения заданное время;
 - г) объект задачи, ожидающей в/в;
 - д) ...

Система ведет списки для каждой подгруппы. Когда необходимо вытеснить объект, он выбирается из наиболее желательного непустого списка.

УПРАВЛЕНИЕ ВНЕШНЕЙ ПАМЯТЬЮ

Важнейшим ресурсом системы является внешняя магнитная память (обычно на магнитных дисках – МД).

Время доступа к конкретной записи данных складывается из:

1. Времени поиска цилиндра t_1 (механическое перемещение магнитных головок на нужный цилиндр).
2. Времени поиска записи t_2 (определяется скоростью вращения дисковых пластин, обычно $t_1 \gg t_2$).
3. Времени чтения-записи данных t_3 (определяется скоростью вращения дисковых пластин).

Необходимость планирования работы МД возникает в мультипрограммных системах с высокой интенсивностью обмена данными, порождающей много запросов на доступ к МД. Чтобы минимизировать время доступа нужно спланировать работу МД в соответствии с некоторым принципом. Планировщик должен:

1. Анализировать позиционные взаимосвязи между ожидающими запросами.
2. Перестраивать очередь запросов так, чтобы их реализация обеспечивалась при минимальных механических перемещениях магнитных головок чтения/записи (МГ) МД.

Различают 2 вида планирования: а) оптимизация по времени поиска цилиндра; б) оптимизация по времени ожидания записи.

Цели и критерии принципов планирования (по множеству запросов).

1. Пропускная способность дискового накопителя $C \rightarrow \max$.
2. Среднее время ответа $T \rightarrow \min$.
3. Дисперсия времени доступа / пропускной способности (мера справедливости, разброса, предсказуемости показателя) $D(T) \rightarrow \min$ / $D(C) \rightarrow \min$.

Планирование улучшает скоростные характеристики в целом, но за счет некоторых отдельных запросов. Критерий №3 – мера того, насколько далеко значения операционной характеристики отдельных запросов могут отклоняться от среднего значения показателя. Если не использовать 3-й показатель, то некоторые запросы могут ждать обслуживания неопределенно долго, а некоторые полностью игнорироваться

Стратегии оптимизации поиска цилиндра:

1. FCFS – «Обслуживание в порядке поступления». Это справедливый метод обслуживания запросов, но характеризуется случайностью поиска – последовательно поступающие запросы на обслуживание могут вызвать длительные подводы считывающих головок от внутренних цилиндров к внешним (и наоборот). Данная стратегия дает небольшую дисперсию, но очень большое значение T и низкую скорость C . Стратегия применяется при низкой нагрузке.
2. SSTF (Shortest-Seek-Time-First) – «Обслуживание с наименьшим временем поиска первым». При позиционировании каретки с МГ чт/зп следующим выбирается запрос, требующий минимального перемещения МГ. Для данной стратегии характерна резкая дискриминация (игнорирование) определенных запросов. Обращения к МД имеют тенденцию к концентрации. В результате запросы далеко отстоящие от центра концентрации игнорируются. Стратегия дает высокую пропускную способность (максимальную), малое среднее время позиционирования, НО высокую дисперсию.

Стратегия используется в системах пакетной обработки, мало пригодна в интерактивных системах из-за плохой предсказуемости т.к. возможно ∞ откладывание запросов.

3. SCAN – «Обслуживание в порядке сканирования». Каретка с МГ совершает движение от внешних цилиндров к внутренним и обратно, обслуживая запросы встречающиеся на пути. Каретка меняет направление движения в том случае, если в текущем направлении больше нет запросов для обслуживания или не достигнет крайнего цилиндра. SCAN аналогичен SSTF, но выбирает запрос, для которого характерно минимальное расстояние поиска в привилегированном направлении. SCAN уступает SSTF и – дает высокую пропускную способность, низкое среднее время ответа, достаточно низкую дисперсию, НО при сканировании МГ на крайних дорожках бывают реже, чем на средних и все еще возможно ∞ откладывание запросов. Используется при малых нагрузках.
4. N-Step SCAN – «Обслуживание в порядке N-шагового сканирования». Это модификация SCAN, в которой при движении МГ обслуживаются только те запросы, которые существовали на момент начала прохода. Запросы, поступающие во время прохода, группируются и упорядочиваются так, чтобы их можно было оптимально обслужить во время обратного хода. Стратегия обеспечивает значения характеристик немного уступающих стратегии SCAN высокую пропускную способность, низкое среднее время ответа, малую дисперсию. Главное достоинство N-Step SCAN состоит в полном исключении ∞ откладывания запросов т.к. при поступлении большого количества запросов к текущему цилиндру они запоминаются для обслуживания при обратном ходе МГ. Используется при очень больших нагрузках.
5. C-SCAN – «Обслуживание в порядке циклического сканирования». Обслуживание запросов производится при движении МГ к внутренним цилиндрам по наикратчайшему времени поиска. Если по направлению движения больше нет запросов, МГ возвращается скачком к началу на обслуживание запроса, ближайшего к внешнему цилиндру и возобновляет выполнение запросов на рабочем ходе каретки с МГ. Данную стратегию можно гибридизировать со стратегией N-Step SCAN (запросы, поступающие во время текущего прямого рабочего хода, обслуживаются при следующем ходе). Гибрид полностью исключает дискриминацию запросов к внутренним или внешним цилиндрам и дает самую малую дисперсию времени ответа. Используется при средних и больших нагрузках.
6. Схема Эшенбаха – «Обслуживание в порядке циклического сканирования только одной дорожки на цилиндре» не зависимо от наличия др. запросов на цилиндре. Предусматривается переупорядочение запросов с учетом углового положения записей. Стратегия находит применение при больших нагрузках в специальных системах.

Оптимизация по времени ожидания записи.

В условиях очень высоких нагрузок растет вероятность одновременного обращения к некоторому цилиндру. В этих условиях целесообразно оптимизация по времени ожидания записи. Стратегия SLTF (Shortest-Latency-Time-First) «Обслуживание с наименьшим временем ожидания первым» – аналог SSTF.

ОРГАНИЗАЦИЯ В/В И ФАЙЛОВЫЕ СИСТЕМЫ

Файл – физическое представление информации о совокупности объектов. Файл состоит из единиц информации – записей. Свойства объекта кодируются в полях записей.

Система Управления Файлами (СУФ) – часть ОС реализующая работу с файлами. Файлы именованные объекты, имя файла определяет функцию отображения адресов в объекты внутри файла.

Требования к СУФ. СУФ должна обеспечивать:

1. Эффективное распределение внешней памяти;
2. Гибкий и многосторонний доступ к данным;
3. Максимальную маскировку механизма реализации от пользователя;
4. Максимальную независимость реализации от типа ЭВМ, ОС, ВнУ (обеспечивает простоту портирования);
5. Возможность совместного использования (разделения) общих файлов;
6. Безопасность и целостность информации, хранимой файлах.
7. Эффективную работу с файлами (согласование скоростей, кэширование).

СУФ хранит информацию о файлах в справочниках:

1. Пользовательское имя файла.
2. Местоположение файла.
3. Уникальный идентификатор файла, одинаковый для всей пользователей.
4. Тип доступа, разрешенный пользователям.
5. Дата возникновения, дата последнего использования.
6. Частота использования.
7. Размер записей.

Запрос на доступ к записям файла предусматривает выполнение следующих действий:

1. По имени файла находится уникальный идентификатор.
2. Определяется местоположение файла.
3. Определяется местоположение записи (собственно доступ к файлу).
4. Генерация команд в/в для аппаратуры.

СУФ имеет многоуровневую структуру:

1. **Система в/в** – нижний уровень, реализует операции в/в, здесь учитывается специфика работы различных внешних устройств. Состоит из:
 - иерархии каналов в/в,
 - устройств управления,
 - ВнУ,
 - программ канала (управляют в/в).

Реализует:

- а) стандартную связь между ЦП и аппаратурой в/в;
 - б) управление ВнУ асинхронными взаимодействующими процессами;
 - в) контроль ошибок в/в;
 - г) локализует на своем уровне специфику работы с ВнУ.
2. **Базисная система управления файлами** – преобразует уникальный идентификатор файла в дескриптор, содержащий физический адрес файла и размер его сегментов, реализует расчленение файлов на подфайлы (директории).
 3. **Логическая система управления файлами** – осуществляет связь уникального идентификатора с пользовательским именем файла (преобразует локальные имена файлов пользователей в системно-ориентированные уникальные идентификаторы), реализует логические команды
 - а) создать файл;
 - б) уничтожить файл;

- в) открыть файл;
- г) закрыть файл;
- д) читать запись;
- е) писать запись.

4. **Методы доступа** – обеспечивают логический доступ к записям файла возможно отличный от физической реализации размещения данных. Различают методы доступа по *виду доступа* и *типу синхронизации*.

По *виду доступа* методы различают на **последовательные** (последовательное чтение и запись данных) и **прямые** (прямая адресация извлекаемых и записываемых данных в файле).

По *типу синхронизации* – на **базисные** (асинхронный доступ по требованию) и **с очередями** (синхронный доступ с опережающей подкачкой записей и буферированием записываемых данных).

Последовательный метод доступа организует записи логического файла в последовательном порядке,

прямой – дает возможность обращаться к записям в произвольном порядке.

В базисном методе доступа пользователь должен сам предусмотреть буферизацию и синхронизацию чтения-обработки-записи данных.

В методе доступа с очередями ОС дает стандартную схему буферизации, но пользователь должен соблюдать соглашения связанные с механизмом буферизации.

Естественным расширением СУФ является СУБД, которая обеспечивает гибкий программный интерфейс для пользователя по организации доступа к экземплярам данных в соответствии с различными логическими структурами.

ПРИНЦИПЫ ОЦЕНКИ ПРОИЗВОДИТЕЛЬНОСТИ ВЫЧИСЛИТЕЛЬНЫХ СИСТЕМ

Цели исследований и показатели производительности.

Различают три цели исследования производительности:

1. Оценка с целью выбора ВС среди альтернатив для приобретения.
2. Оценка производительности разрабатываемой ВС или ее компонент (планирование производительности или проектирование с заданной производительностью).
3. Контроль производительности (сбор и накопление данных о характеристиках ВС для прогноза влияния планируемых изменений конфигурации и ОС на производительность ВС).

В общем случае для оценки производительности необходимо предсказать:

1. Характер прикладных задач, решаемых на ВС,
2. Нагрузку на ВС.

Результаты оценки могут быть использованы для:

1. Наилучшего конфигурирования ВС,
2. Подбора стратегий управления ресурсами ВС,
3. Настройки параметров ОС на требования пользователей.

Среди показателей производительности выделяют следующие:

1. Цикл выполнения задания в пакетных системах (время от момента поступления задания в ВС до его выполнения и возвращения результатов пользователю).
2. Время ответа для интерактивных систем (время от момента ввода данных с клавиатуры до момента начала ответа).
3. Дисперсию времени ответа, являющуюся мерой предсказуемости времени ответа в диалоговых системах.
4. Время реакции в системах реального времени (время от момента ввода данных / наступления события до момента выделения кванта на обслуживание запроса).
5. Пропускную способность (число обслуженных заданий, запросов, транзакций в единицу времени).
6. Загрузку ресурсов ВС.

Для оценки данных показателей используются следующие методы измерений:

1. Измерение элементарных времен, позволяющее оценить техническую производительность компонент и узлов вычислителя.
2. Измерение времени выполнения смеси команд, характерной для применения данной вычислительной установки.
3. Время выполнения образцовой программы, типичной для применения данной ВС (метод часто используется при построении компиляторов для выбора наиболее эффективного кода языковых конструкций программы).
4. Время выполнения измерительных программ, типичных для решаемого на ВС класса задач (обычно данный метод применяется при намерении сменить ВС или ОС для эксплуатации готового программного обеспечения).

5. Время выполнения синтетических программ, совмещающих в себе черты образцовых и измерительных программ.
6. Аналитическое моделирование процессов, протекающих в ВС. Аналитические модели позволяют быстро и наиболее наглядно увидеть достоинства и недостатки ВС, однако формализация вычислительных процессов всегда предусматривает их идеализацию, после чего возникает вопрос адекватности аналитической модели реальной системе и проблема интерпретации полученных результатов.

Анализ системы – Формализация (идеализация) – Решение – Интерпретация результатов

7. Имитационное моделирование ВС, позволяющее исследовать трудно формализуемые при аналитическом моделировании элементы вычислительного процесса.

Набор данных методов позволяет определить

1. Узкие места ВС.
2. Выделить насыщение ресурса (состояния, когда нагрузка на ресурс близка или превосходит его потенциальные возможности).

Пиковая и реальная производительность.

Пиковая (техническая) производительность определяет теоретический максимум быстродействия вычислительной системы в идеальных условиях. **Пиковая производительность определяется числом операций, выполняемых всеми исполнительными устройствами вычислителя**, и равна произведению тактовой частоты, числа арифметическо-логических устройств (конвейеров) процессора и количества процессоров вычислителя. Значение пиковой производительности измеряется в миллионах операций в секунду MIPS (millions instructions per second) или миллионах операций с плавающей точкой в секунду MFLOPS (millions floating operations per second). Пиковая скорость достигается при

1. Бесконечной последовательности несвязанных и не конфликтующих во время доступа в основную память команд,
2. Выборе операндов из внутри кристалльной кэш памяти данных, а команд – из внутри кристалльной кэш-памяти команд.

Пиковая производительность не учитывает функциональное наполнение команд, реализуемое в различных архитектурах процессоров.

Реальная производительность определяется классом решаемых задач. В мировой практике наибольшее распространение получило использование **наборов задач**, характерных для данной области применения. Время выполнения каждой из задач – основа для расчета индекса производительности данной вычислительной системы. При оценке производительности на тестах возникает **три проблемы**, связанные с анализом результатов:

1. Проблема достоверности оценок (корректности методов измерений, выделения показателей, которым можно доверять).

2. Проблема адекватности оценок (выбора тестов, наиболее точно характеризующих производительность при обработке типовых задач).
3. Проблема интерпретации (правильного истолкования результатов тестирования).

Тесты производительности

Существующие тесты можно разбить на **три группы**.

Тесты производителей средств вычислительной техники, используемые для оценивания качества собственных продуктов. Они ориентированы на сравнение компьютеров одного семейства. Одним из таких тестов является индекс iCOMP (Intel Comparative Microprocessor Performance). Индекс iCOMP определяется временем исполнения смеси команд (67% – над 16-разрядными целыми, 3% – над 16 – разрядными числами с плавающей точкой, 25% – над 32-разрядными целыми, 5% – над 32-разрядными числами с плавающей точкой и т.д.). Тесты iCOMP оценивают фактически производительность микропроцессора, а не вычислительной установки. В общем случае тесты производителей являются идеальным средством оценивания быстродействия и технико-экономических показателей вычислительных систем с одной архитектурой, но различными средствами реализации. Существуют тесты производителей и других компонент ВТ.

Стандартные тесты, создаваемые независимыми аналитиками и используемые для сравнения широкого спектра компьютеров. В набор широко применяемых стандартных тестов входит тестовый пакет **Linpack** (автор Джек Донгарра), тестовые наборы **SPEC XX**, разрабатываемые компанией SPEC (Standard Performance Evaluation Corporation), тестовые пакеты **TPC** для оценки производительности серверных платформ, выполняющих в реальном времени транзакции в информационных системах оперативной обработки данных (OLTP системах), данные тесты создаются компанией TPC-C (Transaction Processing Performance Council). В данную группу следует включить также тестовый комплекс WebStone для оценки Web-конфигураций.

Пользовательские тесты, учитывающие специфику конкретного применения вычислительной системы. Данные тесты создаются крупными компаниями, специализирующимися на внедрении компьютерных технологий, и используются для выбора средств вычислительной техники и программного обеспечения под определенные прикладные задачи, поскольку дают наиболее точные оценки производительности заданных приложений. К этому классу можно отнести комплекс тестов NAS, разработанный в исследовательском центре NASA Ames Research Center для оценки производительности многопроцессорных суперкомпьютеров с MPP архитектурой. Центром NASA сформулированы основополагающие требования, которым должны удовлетворять тестовые методики оценки производительности супер вычислителей:

1. Поскольку алгоритмические и программные реализации тестовых программ являются зависимыми от архитектуры и могут отличаться для различных платформ, то тестовые "смеси" должны носить "общий" характер и не следовать какой-либо конкретной архитектуре.
2. Корректность результатов должна быть легко проверяема, что требует точного описания входных и выходных данных и природы вычислений.

3. Используемые вычислительные ресурсы и память должны быть масштабируемыми для повышения производительности.
4. Тексты и спецификации применяемых тестов должны быть доступны и подтверждаться повторной реализацией.

Комплекс тестов NAS общепризнанно считается лучшим тестовым набором для оценки многопроцессорных вычислительных систем с массовым параллелизмом и включает пять расчетных тестов NAS Benchmarks kernel и три теста NAS Benchmarks моделирования реальных задач гидродинамики и аэродинамики. Тесты позволяют оценить вычислительные возможности компьютерной системы и скорость передачи данных между процессорами в массово-параллельных системах. Метрикой теста является относительная производительность по сравнению с показателями традиционного векторного суперкомпьютера, в качестве которого выступает одна из моделей Cray. Тестовый комплекс определяет два класса тестов, отличающихся размерностью вычислений: А и В. Задачи класса В превосходят задачи класса А по размерности в четыре раза. Результаты тестирования в классе А нормируются на производительность однопроцессорного компьютера Cray Y-MP, а класса В – на однопроцессорный Cray C90.

Обычно тесты класса А используются для оценки масштабируемых систем с числом процессорных узлов менее 128. При числе узлов до 512 следует применять тесты класса В.

Комплект тестов NAS Benchmarks kernel включает следующие расчетные задачи.

1. EP (Embarrassingly Parallel). Вычисление интеграла методом Монте-Карло тест "усложненного параллелизма" для измерения вычислительной производительности плавающей арифметики. Тест имеет минимум межпроцессорного взаимодействия и определяет чисто вычислительные характеристики узла при работе с вещественной арифметикой.
2. MG (3D Multigrid). Решение уравнения Пуассона ("Трехмерная решетка") в частных производных. Данный тест требует высокоструктурированной организации взаимодействия процессоров и позволяет оценивать возможности системы выполнять дальние и короткие передачи данных.
3. CG (Conjugate Gradient). Вычисление наименьшего собственного значения больших разреженных матриц методом сопряженных градиентов. Это характерное неструктурированное вычисление на решетке, позволяющее оценить скорость передачи данных на длинные расстояния при отсутствии регулярности.
4. FT (fast Fourier Transformation). Вычисление методом быстрого преобразования Фурье трёхмерного уравнения в частных производных. Данная задача используется как серьезный тест для оценки эффективности взаимодействия удаленных процессоров при передаче данных.
5. IS (Integer Sort). Сортировка целых чисел, позволяющая оценить возможности работы системы с целочисленной арифметикой (в основном одного узла) и потенциал вычислителя при выполнении межпроцессорного взаимодействия.

Комплекс тестов NAS Benchmarks для задач моделирования включает следующие модули.

1. LU (LU Solver). Решение системы уравнений с равномерно разреженной блочной треугольной матрицей 5x5.
2. SP (Scalar Pentadiagonal). Решение нескольких независимых систем скалярных уравнений пентадиагональные матрицы с преобладанием недиагональных членов.
3. BT (Block Tridiagonal). Решение серии независимых систем уравнений блочные трехдиагональные матрицы 5x5 с преобладанием недиагональных элементов.

Следует отметить, что результаты тестирования по различным тестам обычно не сопоставимы. Наиболее широкое распространение в настоящее время получили наборы стандартных тестов.

Тесты Linpack представляют собой совокупность программ решения задач линейной алгебры. Параметрами тестов являются

- а) порядок матрицы;
- б) формат значений элементов матрицы, определяющий точность представления ее элементов (одинарная/двойная);
- в) способ компиляции (с оптимизацией либо без нее);
- г) возможность применения оптимизированной библиотеки стандартных функций.

Для больших размерностей обрабатываемых матриц вычислительные системы, как правило, дают реальную производительность в интервале 80-95% от пикового показателя. По результатам тестирования набором тестов Linpack с 1993 года ведется список TOP 500, включающий 500 самых производительных вычислительных систем в мире. В основном этот список содержит уникальные многопроцессорные MPP-системы, однако в последние годы в конец этого списка стали попадать и коммерческие вычислители с SMP-архитектурой (особенно на фоне роста количества ядер на кристалле ЦП).

Тесты SPEC XX. В данной группе тестов существует несколько тестовых пакетов: SPEC89, SPEC92, SPEC95, SPEC98, SPEC2000 и т.д. Далее эволюция тестов.

Пакет SPEC89 включает два тестовых набора Cint89, состоящий из **четырёх** программ целочисленной обработки, и Cfp89, объединяющий **шесть** программ со значительным объемом операций над числами с плавающей точкой двойной точности. Все десять программ представляют собой достаточно сложные коды на языках C и Fortran с широким спектром решаемых задач от оптимизации представлений функций булевой логики в программируемых логических схемах до моделирования замещения атомов в квантовой химии.

Методика оценки производительности SPEC89 предполагает формирование десяти дифференциальных оценок SPECratio(i), каждая из которых определяется как отношение времени выполнения i-й программы на тестируемом компьютере ко времени выполнения той же программы на вычислителе DEC VAX 11/780 (нормирующий показатель). Интегральной характеристикой производительности компьютера является показатель SPECmark, являющийся средним геометрическим всех десяти частных оценок SPECratio.

К параметру SPECmark добавляются еще две оценки SPECint89 и SPECfp89, отдельно характеризующие быстродействие компьютера при обработке целочисленных данных и вещественных чисел. Принцип расчета этих показателей такой же, как и параметра SPECmark: SPECint89 представляет собой среднее геометрическое частных оценок SPECratio для четырех программ из набора Cint89, а SPECfp89 – аналогичную величину для шести программ из состава Cfp89. Пакет тестовых программ SPEC92 расширяет набор тестируемых функций по сравнению со SPEC89.

$$SPEC\ int\ XXXX = \left(\prod_{i=1}^n SPEC\ int\ ratio(i) \right)^{1/n}, \quad SPEC\ fp\ XXXX = \left(\prod_{j=1}^m SPEC\ fp\ ratio(j) \right)^{1/m},$$

$$SPEC\ mark\ XXXX = \left(\prod_{i=1}^n SPEC\ int\ ratio(i) \times \prod_{j=1}^m SPEC\ fp\ ratio(j) \right)^{1/(n+m)}.$$

n – количество тестовых программ обработки целочисленных данных;

m – количество тестовых программ обработки данных с плавающей точкой.

Пакет оценивающих программ Cint92 предназначен для оценки производительности вычислительных систем при выполнении целочисленных операций преимущественно в коммерческой области применения. В его состав входят шесть эталонных тестов, написанных на языке C и представляющих собой:

- задачу из теории сетей;
- интерпретатор языка Lisp;
- задачу логического проектирования;
- Unix-утилиту упаковки текстового файла размером 1 Мбайт, который 20 раз подвергается сжатию;
- операции со строками и столбцами электронной таблицы;
- компилятор языка C.

Пакет оценочных программ Cfp92 предназначен для оценки производительности ВС при выполнении операций с плавающей точкой преимущественно в технической и научной областях применения. Он включает 14 прикладных программ, две из которых написаны на языке C и 12 – на языке Fortran. В пакет входят:

- программы схемного проектирования;
- модули моделирования термодинамики ядерного реактора методом Монте-Карло;
- задачи квантовой химии и физики;
- алгоритмы решения уравнений Максвелла;
- преобразование координат;

- трассировка оптических лучей;
- задачи робототехники и нейросетей;
- моделирование человеческого уха;
- решение уравнений Навье-Стокса для определения параметра межгалактического газа;
- семь библиотечных функций обработки матриц (умножение, обращение и т.д.).

В SPEC92 введено одно качественное новшество – **показатель SPECrate**, характеризующий мультипрограммную обработку задач методом однородной нагрузки в многопроцессорной ВС. При этом тестируемая ВС выполняет множество копий одной программы, а показателем производительности многопроцессорной обработки служит количество копий, завершенных за определенный интервал времени.

SPECrate(i)=количество копий i-й программы, выполненных на заданное время.

Для получения оценки SPECrate используются те же программы, что и для расчета показателей SPECint92 и SPECfp92. Тестовый модуль реализуется как несколько копий, образующих одно задание, а результатом измерений является нормированное общее время выполнения всех копий задания. Данной процедуре подвергается каждая из 20 тестовых программ, что позволяет получить шесть частных оценок SPECratio для программ целочисленной обработки и 14 – для программ вещественных чисел. Эти показатели позволяют получить представление о:

- **возможностях компилятора по организации параллельного мульти задачного кода;**
- **способностях операционной системы организовать эффективное динамическое распределение ресурсов ВС между выполняемыми параллельными программами.**

Тестовые программы SPEC95 используют два тестовых набора Cint95 и Cfp95, состоящие из 8 и 10 программ соответственно. При испытаниях ВС формируются:

- **индексы производительности SPECint95, SPECfp95, SPECint base95 и SPECfp base95 для фиксированной и плавающей точки в режиме компиляции с агрессивной и консервативной оптимизацией** соответственно;
- **индексы пропускной способности SPECint rate95, SPECfp rate95 и SPECint rate base95, SPECfp rate base95 для оценки многозадачных режимов SMP-систем с агрессивной и консервативной оптимизацией** соответственно.

Все интегральные индексы производительности формируются как среднее геометрическое индексов по отдельным тестам.

Комплект процессорных тестов SPEC2000 включает:

19 приложений, не входивших ранее ни в один из тестовых наборов SPEC, и предназначен для **тестирования главным образом трех компонент вычислителя: процессора, иерархической памяти, компиляторов.** В комплект тестов SPECint2000 входит 12 программ:

- 2 программы компрессии;
- размещение элементов и расчет разводки FPGA-микросхем;
- компилятор Си;
- комбинаторная оптимизация;
- шахматная игра;
- обработка текста;
- компьютерная визуализация;
- интерпретатор perl;
- интерпретатор теории групп;
- объектно-ориентированная база данных;
- моделирование размещения элементов и разводки микропроцессора.

Тестовый набор SPECfp2000 включает 14 программ:

- физика (квантовая хромо динамика);
- моделирование мелкой воды;
- многосеточные методы (трехмерное потенциальное поле);
- дифференциальные уравнения в частных производных;
- библиотека трехмерной графики;
- вычислительная гидродинамика;
- распознавание образов/нейросети;
- моделирование распространения сейсмических волн;
- обработка изображений (распознавание лиц);
- вычислительная химия;
- теория чисел (поиск простых чисел);
- моделирование столкновений с применением метода конечных элементов;
- проектирование ускорителей элементарных частиц;
- метеорология (распространение загрязняющих веществ).

Недавно появился тест SPECWeb.

Пакет тестовых программ TPC применяется для оценки производительности при работе с базами данных. Тесты TPC включают набор тестов для измерения эффективности

функционирования в различных режимах обработки данных – TPC-A, TPC-B, TPC-C, TPC-D. Тесты дают сравнительную оценку стоимости и производительности совокупности аппаратно-программных средств (Средств вычислительной техники, ОС, СУБД, мониторов транзакций). С 2000 года пакет тестовых программ TPC включен тест TPC-W. В основе тестового набора TPC лежат следующие фундаментальные принципы:

- производительность соотносится с общей стоимостью тестируемой системы, включая аппаратную составляющую, программное обеспечение и эксплуатационные издержки в расчете на заданный срок эксплуатации;
- тест формулируется на высоком функциональном уровне без связи с конкретной программно-аппаратной платформой, допуская тем самым сравнение между собой различных реализаций;
- тест должен учитывать потенциальный рост мощности вычислительных систем и допускать масштабирование нагрузки по числу пользователей и объему базы данных.

Тест TPC-A используется для оценки эффективности исполнения **транзакций типа «дебет-кредит»**. При этом моделируется работа филиальной сети банка по приему-выдаче вкладов клиентов (*операционный день*). В тесте используется один тип транзакции, задающий манипулирование со счетом клиента (так называемые "короткие транзакции"). Ведется история реализации транзакции. Доступ к центральному серверу баз данных осуществляется либо по терминальным линиям, либо по сети с удаленных рабочих мест. **Соотношение СТОИМОСТЬ/ПРОИЗВОДИТЕЛЬНОСТЬ** оценивается как стоимость 5-летней эксплуатации всего полностью сконфигурированного оборудования и программных средств ВС, отнесенная к максимальному числу транзакций в секунду, которое способна выполнить система.

Тест TPC-B аналогичен тесту TPC-A, но ориентирован на **пакетный режим обработки** и имеет другую методику подсчета стоимости ВС. **Стоимость системы подсчитывается без стоимости терминалов, терминальных линий связи и сетевых соединений.** Задачей теста является **оценка эффективности работы сервера, дисковой конфигурации и СУБД в условиях интенсивной нагрузки.** В настоящее время данный тест сохранил лишь историческое значение.

Тест TPC-C является основным продуктом TPC и предназначен для оценки **эффективности ВС при работе в режиме оперативной обработки транзакций (On-Line Transaction Processing).** Тест составляет **смесь транзакций, характерных для OLTP, считывающих и обновляющих содержимое базы данных (смесь «коротких» и «длинных» транзакций).** При исполнении теста TPC-C моделируется функционирование нескольких складов компании. Каждый склад имеет 10 участков, каждый участок – 3000 заказчиков, 10000 наименований товаров. **Используются операции агрегирования, соединения, селекции отношений базы данных.** Используется пять типов транзакций: новый заказ, платеж, статус заказа, выполнение заказа, уровень запасов по последним 20 наименованиям запрошенных товаров. Тест оценивает количество транзакций, выполненных в минуту $tpmC$ (transaction per minute C), и стоимость одной выполненной транзакции в минуту.

Тест TPC-D используется для оценки эффективности сложных информационных систем поддержки принятия решений (Decision Support System) с много направленными соединениями, сортировками и агрегированием данных отношений. Создаваемая тестом нагрузка существенно отличается от характерного для OLTP потока простых транзакций, обрабатывающих небольшие порции данных. Системам поддержки принятия решений присуще небольшое число сложных взаимосвязанных запросов, обрабатывающих обычно всю базу данных. На результаты тестирования важнейшее влияние оказывает **объем базы данных**. В тесте предусмотрено 6 градаций объемов базы данных: 1 Гбайт, 10 Гбайт, 30 Гбайт, 100 Гбайт, 300 Гбайт, 1 Тбайт. По результатам тестирования определяются:

- производительность обработки запросов QppD (Query processing Performance), измеряемое количеством запросов, обработанных при монопольном использовании всех ресурсов ВС;
- пропускная способность системы QthD (Query Throughput), измеряемое количеством запросов, обрабатываемых в течение часа;
- отношение стоимости к производительности QphD, измеряемое как стоимость 5-летней эксплуатации системы, отнесенная к числу запросов, обработанных за час.

В настоящее время тест TPC-D модернизирован и разделен на два теста: TPC-R (business reporting) и TPC-H (ad hoc querying). Первый тест ориентирован на подготовку регулярных отчетов, а второй – на составление нерегулярных, непрогнозируемых отчетов. Тест TPC-R служит для оценки производительности систем поддержки бизнеса и подготовки отчетов (Business Support and Report workloads). Тест имеет две метрики: QphR (TPCR Composite Query-Per-Hour Performance) и долл./QphR.

Тест TPC-H оценивает производительность систем принятия решений при выполнении набора запросов к стандартной базе данных. Основной метрикой является QphH (TPCH Composite Query-Per-Hour Performance), которая публикуется совместно с указанием размера базы данных, изменяющейся в пределах от 1 до 10 тыс. Гбайт. Вторая метрика «цена/производительность» (TPC-H Price/Performance Metric), измеряемая в долл./QphH, также соотносится с размером базы данных.

Тест TPC-W предназначен для оценки систем поддержки электронной коммерции, электронного бизнеса, коммерческих Web-приложений, корпоративных сетей. TPC-W измеряет производительность и производительность, отнесенную к цене, аппаратно-программного обеспечения для транзакционной Web-среды. От близких по смыслу тестов SPECWeb и WebStone тест TPC-W отличается тем, что включает доступ к динамически генерируемым базам данных Web-страниц, функции обеспечения безопасности пользовательского интерфейса и внешних транзакций, а также позволяет оценивать экономическую эффективность решения в расчете на трехлетний эксплуатационный срок. Основной моделью тестирования сайтов является книжный магазин, представленный восемью таблицами. Размер базы данных может изменяться от 1000 до 1 млн товаров. **Базовые метрики – число взаимодействий (покупок) с Web-сервером в секунду (Web Interactions Per Second – WIPS) с делением по категориям баз данных различных размеров и отношение стоимости по категориям баз данных к WIPS (Price Per WIPS),**

измеряемое в долл./WIPS. Дополнительные метрики WIPsb (WIPS browsing) и WIPso (WIPS ordering) позволяют оценивать скорость навигации и оформления заказов соответственно.

Тест WebStone используется для оценки конфигураций Web. Тест WebStone позволяет дать объективную **оценку аппаратуры, программного обеспечения и дисциплины взаимодействия с сетью**. Данный тест отражает специфику работы в глобальной сети с многократными переключениями, коррекцией ошибок, переадресациями и т.д. Он позволяет моделировать разнородную среду, в которой работает одновременно множество клиентов, порождающих разнообразные процессы доступа к Web-информации различных серверов. Тест представляет собой **инструмент измерения загрузки серверов в зависимости от режима работы клиентов, задающих HTTP-трафик**. Предусмотрены четыре моделирующие смеси:

1. Тестовый набор для моделирования работы с сетью через аналоговый модем, файлы данной смеси содержат небольшие, как правило, текстовые страницы объемом до 20 Кбайт.
2. Тестовый набор для моделирования работы клиентов ЛВС при обмене файлами размера 1-1000 Кбайт.
3. Тестовый набор для моделирования полнофункциональной работы с сетью, включающий мультимедийную информацию (видео, звук, графику) объемом до нескольких Мбайт.
4. Четвертый тестовый набор объединяет первую и третью смеси.

Основными метриками теста WebStone являются пропускная способность и время выполнения запроса (задержка), которые усредняются по множеству измерений за сеанс тестирования. Различают два типа задержки: **время установления соединения между клиентом и сервером и время передачи данных (исполнения запроса)**. Для оценки скорости безошибочного обслуживания максимального и заданного множества клиентов тестом WebStone порождаются запросы серверу по протоколу HTTP и организуется обработка данных по мере их поступления. При этом **проводится измерение среднего и максимального времени соединения, среднего и максимального времени отклика, пропускной способности, количества обработанных страниц, числа открытых файлов**. Тест WebStone организован в виде распределённого набора прикладных процессов – головного WebMASTER и потомков WebChildren. Стандартную тестовую смесь можно настроить с помощью следующих параметров:

1. Продолжительность выполнения теста, задаваемая в минутах, максимальное значение которого определяется числом потомков и объемом памяти, выделяемым для каждого клиента.
2. Число повторений, позволяющее устранить элемент случайности и выявить устойчивые закономерности.
3. Количество тестовых файлов.
4. Число страниц в формате HTML.
5. Опции программного и аппаратного обеспечения сервера.
6. Количество потомков.
7. Количество сетей, управляемых одним сервером.

8. Число клиентов, моделирующее режимы загрузки ресурсов.
9. Загрузка страниц, задающая активность их использования.
10. Ведение журнала, позволяющее получить подробный протокол работы теста.
11. Отладка – для диагностики возможных сбоев.

При описании операционного окружения для работы теста необходимо задать конфигурацию программ и аппаратуры, имитирующую различные реальные сетевые комплексы.

ЗАЩИТА ОБЪЕКТОВ ОПЕРАЦИОННЫХ СИСТЕМ

В общем случае включает ЗА

- Аутентификация (идентифицирует субъекта, попадающего в систему=КТО),
- Авторизация (определяет для субъекта разрешенные действия в системе=ЧТО МОЖНО ДЕЛАТЬ),
- Аудит (фиксация в системном журнале событий, связанных с доступом к защищаемым ресурсам системы=ЧТО СДЕЛАНО).

Защита ОС – механизмы защиты объектов системы от их окружения. Механизмы различны для различных частей ОС. Вся информация в ВС расположена на различных устройствах памяти. Задача защиты – организовать защиту информации разделив пространство адресов этих устройств на области и разрешить только контролируемый обмен информацией между областями. Каждая область должна быть защищена от незаконных попыток получить / изменить информацию со стороны внешнего окружения. Здесь возникает 2 проблемы:

- эффективность охраны,
- эффективность доступа.

Области адресуемого пространства должны быть ограждены **стенами с воротами** для обеспечения коммуникационных связей. **Стены** должны быть связаны с механизмом **адресации**, адресные ссылки внутри области не должны требовать обширных проверок, **адресные ссылки на объекты вне данной области** должны реализовываться через механизм – **ворота**. Коммуникационные связи (ворота) должны охраняться механизмом контроля, который разрешал бы доступ только субъектам, имеющим на это право.

Способы возведения стен: **права доступа, обеспечение секретности пути к объекту, применение методов кодирования.**

Права доступа. При сегментированном пространстве адресов каждый адрес внутри сегмента проверяется на принадлежность диапазону от начального адреса до начального адреса + длина сегмента. Т. о. адресный механизм возводит **Стену** вокруг всех адресов внутри сегмента и **Ворота**. Сами сегменты защищаются через таблицу сегментов при проверке законности

обращения (в соответствии со статусом защиты) со стороны процесса, выдавшего ссылку.

Аналогична защита файлов – при попытке процесса получить доступ к файлу, его запрос проходит через СУФ, которая проверяет права процесса на соответствие статусу защиты. Не должно быть способов получения доступа к файлу минуя **СУФ=ворота**.

Обеспечение секретности пути к объекту – другой способ защиты. Объект не защищен, но его местоположение известно только определенным привилегированным **процессам – проводникам=ворота**. Этот механизм не является абсолютно непроницаемым. Пользователь может по ошибке или намеренно получить доступ к объекту и повредить его.

Методы кодирования. Информация расположена в незащищенных областях, но в закодированном виде. Для раскодирования процесс должен знать ключ (известный только привилегированным процессам) к коду.

Ворота=код + ключ к нему + привилегированные процессы.

При любом способе организации защиты объектов ОС наиболее предпочтительным механизмом являются **мониторы защиты**, выполняющие роль ворот. Мониторы, обеспечивающие синхронизацию, могут быть расширены для обеспечения защиты. Мониторы могут иметь программно-техническую реализацию.

СУФ – программный монитор.

Механизм логической адресации – имеет аппаратную поддержку.

Каждый объект может иметь собственный монитор защиты. Но с точки зрения эффективности целесообразно для похожих объектов иметь общие мониторы. Решение: разбиение управления защитой по группам объектов. Каждый монитор защиты служит воротами для большого класса сходных объектов (например, СУФ – требует соблюдения правил доступа для класса объектов – файлы).

Домены и возможности

В вопросах защиты ОС различают:

- активные субъекты (обычно процесс);
- пассивные объекты (обычно фрагменты памяти, файлы).

Субъект и объект с течением времени могут меняться ролями.

Часто различные субъекты получают одинаковые права в отношении различных объектов. Полезно выделить классы прав доступа в отдельные категории – домены. **Домен определяет контекст защиты, по которому может действовать некоторый субъект (права доступа).** Процесс меняет свой статус защиты, меняя домен в котором он работает. Разделение статуса защиты на домены позволяет обращаться с правами доступа независимо от субъектов (это гибкое средство для совместного использования механизмов защиты).

Объекты, субъекты и домены – именуются уникальными идентификаторами. Это дает возможность монитору защиты контролировать законность прав доступа. **Конкретное право доступа некоторого субъекта в отношении некоторого объекта – возможность** (например, доступ только для чтения в отношении некоторого файла). Одна из **проблем защиты – изменение прав доступа.** Решение проблемы: процесс, раздающий права, имеет возможность их отбирать.

Описание статуса защиты

В произвольный момент времени имеется:

Множество доменов $X=\{x(1),x(2),\dots,x(n)\}$.

Множество объектов $Y=\{y(1),y(2),\dots,y(m)\}$.

Статус защиты описывается функциональным отображением **F: $X*Y \rightarrow A$** , множество **$A=\{A1,...,AN\}$** – множество атрибутов доступа (только чтение, чтение и запись, только выполнение, ...). Домены рассматриваются независимо от субъектов, в любой момент времени существует способ связать заданного субъекта с определенным доменом. Статус защиты не статичен! Домены и объекты непрерывно создаются, уничтожаются, изменяются. Субъекты меняют домены, в которых они действуют.

Если отображение **F** зависит только от значений $x(i)$ и $y(j)$, то статус защиты хорошо описывается **матрицей**, строки которой задают имена доменов, а столбцы – имена объектов в системе. Для каждого домена и каждого объекта матрица определяет права данного домена на данный объект (в виде атрибутов доступа). **Обычно домен имеет права по отношению к небольшому числу объектов системы, тогда матрица содержит большое число пустых позиций.**

Матричная модель может быть обобщена до функциональной, если в качестве **элементов матрицы использовать процедуры** (функции от др. переменных), проверяющие законность доступа к объекту.

В матричной схеме защиты должны запрещаться бесконтрольные изменения элементов матрицы (в противном случае не будет защиты). Но права доступа должны изменяться и для защиты изменений нужны специальные меры. **Защиту изменений можно реализовать обращаясь с доменами как с объектами.** Матрицу необходимо расширить, включив в нее столбцы, соответствующие доменам и изменять их!!!

Для реализации монитором защищенных изменений каждому элементу матрицы (каждому праву доступа) приписываются дополнительные специальные атрибуты: **владелец, управление, признак копирования.**

Атрибут **управление** какого-либо домена $x(i)$ над другим доменом $x(j)$ означает, что домен $x(i)$ полностью контролирует домен $x(j)$. Возможна иерархия доменов (например, домен сисадмина контролирует все пользовательские домены).

Атрибут **владелец** домена $x(i)$ по отношению к объекту y означает, что объект y принадлежит домену $x(i)$.

Атрибут **признак копирования** разрешает копирование права доступа в другой домен.

Статус защиты системы (сужение и расширение прав доступа к объектам системы в различных доменах) управляется правилами изменения элементов матрицы.

Правила расширения прав.

Правило 1. Домен $x(i)$ может передавать атрибуты доступа элементу $A(x(j),y)$, устанавливая или нет **признак копирования**, если домен $x(i)$ имеет атрибут **владелец** по отношению к объекту y .

Правило 2. Домен $x(i)$ может скопировать атрибут доступа A_k в элемент $A(x(j),y)$, если атрибут A_k находится в соответствующем элементе $A(x(i),y)$ и для него установлен атрибут **признак копирования**.

Правила сужения прав.

Правило 3. Домен $x(i)$ может удалить атрибуты доступа из элемента $A(x(j),y)$, если он имеет атрибут **управление** над доменом $x(j)$.

Правило 4. Домен $x(i)$ может удалить атрибуты доступа из элемента $A(x(j),y)$, если домен $x(i)$ имеет атрибут **владелец** по отношению к объекту y и, кроме того, домен $x(j)$ не имеет защищенного доступа к объекту y . Домен $x(j)$ принимает предохранительные меры против отмены его прав, отказываясь признать какие либо права доступа, которые не защищены.

Данную схему и набор правил для практической реализации необходимо дополнить:

1. Связями субъектов с доменами.
2. Перемещениями субъектов от одного домена к другому.
3. Возможностью создания новых доменов.
4. Возможностью удаления ненужных объектов или доменов из описания статусов защиты.

Схемы реализации защиты

Задача – найти компромисс между временными накладными расходами (определяются организацией структуры данных, описывающей схему защиты), издержками памяти, гибкостью системы защиты.

1. Для представления статуса защиты используется **матрица**. Тогда в таблице T описание статуса защиты состоит из триплетов [**домен, объект, атрибуты доступа**]. Поиск элемента $A(x,y)$ ведется по глобальной сильно разреженной таблице T .

Недостатки схемы:

- а) Таблица T очень велика и вся не умещается в памяти (возможно хранение только отдельных фрагментов T).
- б) Трудно осуществлять объединение объектов или доменов в группы.
- в) сложно реализовать поиск объектов, к которым некоторый домен имеет доступ или по отношению, к которым он является владельцем.

2. Для представления статуса защиты используется **список возможностей** (строка матрицы). Все объекты, к которым имеет доступ некоторый домен включают в список, относящийся к данному домену – $[y,A(x,y)]$ – это возможности, соответствующие правам доступа домена x к объекту y . Такие списки можно хранить в области памяти, защищенной ключом с доступом через высоко привилегированный монитор защиты. Удобное решение для пользователей ОС.

3. Для представления статуса защиты используется **список управления доступом** (столбец матрицы). Все домены, имеющие доступ к конкретному объекту, объединяются в список – $[x,A(x,y)]$ – элементы списка задают

атрибуты доступа для каждого домена, имеющего права доступа к объекту *у*. Для каждого объекта создавший его субъект задает процедуру и список управления доступом. Процедура отвечает за проверку доменов, которые пытаются получить доступ к данному объекту. Удобное решение для администратора ОС.

4. Для представления статуса защиты используется **механизм замок-ключ** с уникальными битовыми наборами. **В каждом домене** имеется список объектов с битовым набором для каждого объекта, битовый набор (ключ) служит для идентификации прав доступа к данному объекту. **С каждым объектом** связывается список уникальных битовых наборов (замков) для каждого из которых указаны права доступа. Для получения доступа к объекту субъект предъявляет ключ монитору защиты, охраняющему данный объект. Монитор устанавливает соответствие ключа и замка, связанного с данным объектом и разрешает/запрещает доступ в зависимости от требуемого вида доступа. Схема **замок-ключ** эффективна и гибка:

- а) с битовыми наборами, соответствующими возможностям можно действовать независимо от значений наборов;
- б) отмена и изменение прав доступа осуществляется изменением списка битовых наборов (замков), связанного с данным объектом.

Недостатком является необходимость поддержки длинных битовых наборов (для исключения повторения битовых наборов).

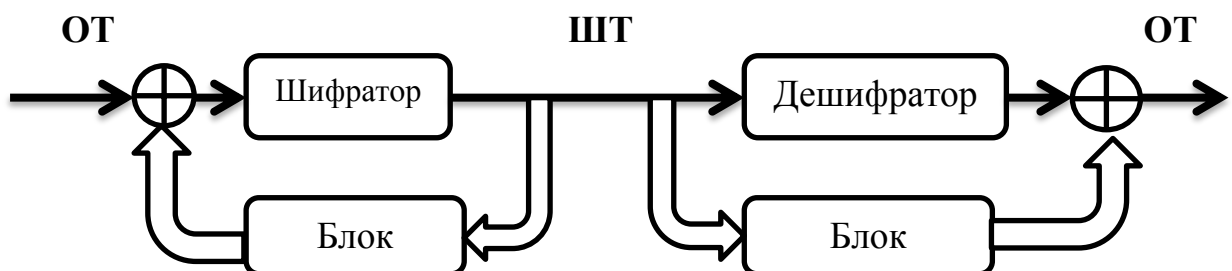
Схемы 2-4 более выигрышны по быстродействию, т.к. проверки законности доступа ими ограничиваются более локализованным поиском.

Криптосистема

Криптосистема выполняет 2 преобразования – шифрование (Ш) и дешифрование (Д).

Критерий качества криптосистемы: сохранение свойств секретности при общеизвестной сущности алгоритмов Ш и Д. Это достигается с помощью ключа К. Различают симметричные (с одним ключом, DES) и несимметричные (с двумя ключами – открытым и секретным, RSA) алгоритмы шифрования. Применяется обычно блочное кодирование.

В **симметричных** алгоритмах шифрования (отличающихся относительно невысокой трудоемкостью) существует опасность, что повторяющиеся группы данных открытого текста (ОТ) будут явно выражены в зашифрованном тексте (ШТ). Тогда по зашифрованному тексту и известным фрагментам открытого текста возможно восстановление ключа. Решением данной проблемы является метод сцепления или режим шифрования с обратной связью. Суть данного метода состоит в том, чтобы включить в первый кодируемый блок случайное число, а далее шифровать блоки в зависимости от значений предыдущих блоков.



Шифрование:

Перед шифрованием каждого блока ОТ к нему прибавляется по модулю 2 (исключающее или) содержание предыдущего зашифрованного блока (**по обратной связи**). В результате очередной зашифрованный блок зависит от содержимого всего предыдущего ОТ.

$Ш_i = E(O_i + Ш_{i-1})$, где O_i – i-й блок открытого текста, $Ш_i$ – i-й блок зашифрованного текста, E – функция шифрования.

Дешифрование:

$$O_i = E^{-1}(Ш_i) + Ш_{i-1}.$$

При дешифровании дешифруется очередной зашифрованный блок и складывается по модулю 2 с предыдущим зашифрованным блоком (**по прямой связи**).

Т. о. любое изменение ОТ многократно повторяется в ШТ!

НО ошибки, возникающие при передаче ШТ по линиям связи – локализуются только в 2-х блоках: 1) в том блоке, где произошла ошибка ($i-m$), 2) в следующем за ним блоке ($i+1-m$).

Несимметричные алгоритмы шифрования основаны на необратимых/односторонних (трудно обратимых, обратимых с очень высокой трудоемкостью) функциях. Примером несимметричного крипто алгоритма является метод RSA (Rivest, Shamir, Adleman), основанный на необратимости функции возведения в степень по модулю большого числа. Суть метода состоит в следующем:

1. Случайно выбираются 2 очень больших ПРОСТЫХ числа p и q .
2. Вычисляются 2 произведения $n=p*q$ и $m=(p-1)*(q-1)$.
3. Выбирается случайное целое число E , не имеющее общих сомножителей с m .
4. Находится целое число D , такое, что $D*E=1$ по модулю m .
5. Исходный открытый текст ОТ разбивается на блоки X длины менее n : $0 < X < n$.
6. Для шифрования блока сообщения необходимо вычислить $c = x^E$ по модулю n .
7. Для дешифрования вычисляется $x = c^D$ по модулю n .

Таким образом, для шифрования сообщения необходимо знать пару чисел (E,n) , а чтобы дешифровать – пару чисел (D,n) . При шифровании первая пара – это открытый ключ, а вторая – закрытый.

Зная открытый ключ (E,n) , можно вычислить значение закрытого ключа D . Необходимым промежуточным действием в этом преобразовании является нахождение чисел p и q , для чего нужно разложить на простые множители очень большое число n , на что требуется очень много времени. Огромная вычислительная сложность разложения большого числа на простые множители обеспечивает высокую криптостойкость алгоритма RSA.

Программная реализация крипто алгоритмов типа RSA существенно сложнее и значительно менее производительна, чем реализация классических крипто алгоритмов типа DES. Обусловлено это сложностью реализации модульной

арифметики (с высокой разрядностью обрабатываемых чисел). Обычно RSA используют для:

1. Шифрования небольших объемов информации;
2. Рассылки классических секретных ключей крипто алгоритмов типа DES;
3. Реализации алгоритмов цифровой подписи.

Сравнительные характеристики алгоритмов шифрования

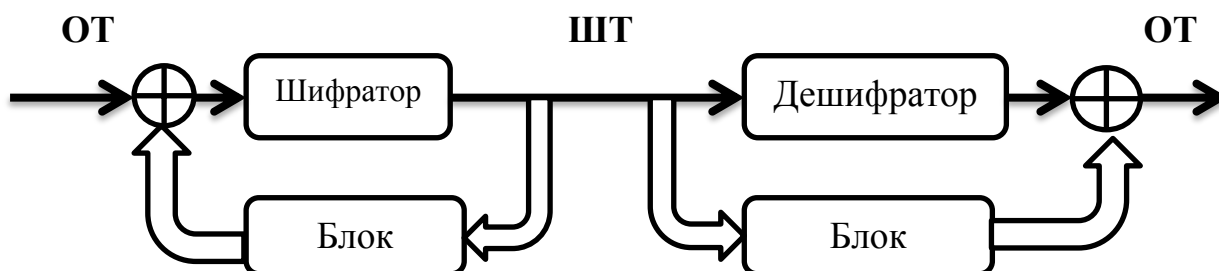
Характеристика	DES	RSA
Скорость шифрования	Высокая	Низкая
Используемая функция шифрования	Перестановка и подстановка	Возведение в степень
Длина ключа	56 бит	Более 500 бит
Наименее затратный крипто анализ (его сложность определяет стойкость алгоритма)	Перебор по всему ключевому пространству	Разложение числа на простые множители
Время генерации ключа	Миллисекунды	Минуты
Тип ключа	Симметричный	Асимметричный

ЛЕКЦИЯ 9. КРИПТОГРАФИЧЕСКИЕ МЕТОДЫ ЗАЩИТЫ

Криптосистема выполняет 2 преобразования – шифрование (Ш) и дешифрование (Д).

Критерий качества криптосистемы: сохранение свойств секретности при общеизвестной сущности алгоритмов Ш и Д. Это достигается с помощью ключа К. Различают симметричные (с одним ключом, DES) и несимметричные (с двумя ключами – открытым и секретным, RSA) алгоритмы шифрования. Применяется обычно блочное кодирование.

В **СИММЕТРИЧНЫХ** алгоритмах шифрования (отличающихся относительно невысокой трудоемкостью) существует опасность, что повторяющиеся группы данных открытого текста (ОТ) будут явно выражены в зашифрованном тексте (ШТ). Тогда по зашифрованному тексту и известным фрагментам открытого текста возможно восстановление ключа. Решением данной проблемы является метод сцепления или режим шифрования с обратной связью. Суть данного метода состоит в том, чтобы включить в первый кодируемый блок случайное число, а далее шифровать блоки в зависимости от значений предыдущих блоков.



Шифрование:

Перед шифрованием каждого блока ОТ к нему прибавляется по модулю 2 (исключающее или) содержание предыдущего зашифрованного блока (**по обратной связи**). В результате очередной зашифрованный блок зависит от содержимого всего предыдущего ОТ.

$Ш_i = E(O_i + Ш_{i-1})$, где O_i – i-й блок открытого текста, $Ш_i$ – i-й блок зашифрованного текста, E – функция шифрования.

Дешифрование:

$$O_i = E^{-1}(Ш_i) + Ш_{i-1}.$$

При дешифровании дешифруется очередной зашифрованный блок и складывается по модулю 2 с предыдущим зашифрованным блоком (**по прямой связи**).

Т.о. любое изменение ОТ многократно повторяется в ШТ!

НО ошибки, возникающие при передаче ШТ по линиям связи – локализуются только в 2-х блоках: 1) в том блоке, где произошла ошибка ($i-m$), 2) в следующем за ним блоке ($i+1-m$).

НЕСИММЕТРИЧНЫЕ алгоритмы шифрования основаны на необратимых/односторонних (трудно обратимых, обратимых с очень высокой трудоемкостью) функциях. Примером несимметричного крипто алгоритма является метод RSA (Rivest, Shamir, Adleman), основанный на необратимости функции возведения в степень по модулю большого числа. Суть метода состоит в следующем:

1. Случайно выбираются 2 очень больших **ПРОСТЫХ** числа **p** и **q**.
2. Вычисляются 2 произведения **$n=p*q$** и **$m=(p-1)*(q-1)$** .
3. Выбирается случайное целое число **E**, не имеющее общих сомножителей с **m**.
4. Находится целое число **D**, такое, что **$D*E=1$** по модулю **m**.
5. Исходный открытый текст ОТ разбивается на блоки **X** длины менее **n**: **$0 < X < n$** .
6. Для шифрования блока сообщения необходимо вычислить **$c = x^E$** по модулю **n**.
7. Для дешифрования вычисляется **$x = c^D$** по модулю **n**.

Таким образом, для шифрования сообщения необходимо знать пару чисел (**E,n**), а чтобы дешифровать – пару чисел (**D,n**). При шифровании первая пара – это открытый ключ, а вторая – закрытый.

Зная открытый ключ (**E,n**), можно вычислить значение закрытого ключа **D**. Необходимым промежуточным действием в этом преобразовании является нахождение чисел **p** и **q**, для чего нужно разложить на простые множители очень большое число **n**, на что требуется очень много времени. Огромная вычислительная сложность разложения большого числа на простые множители обеспечивает высокую криптостойкость алгоритма RSA.

Программная реализация крипто алгоритмов типа RSA существенно сложнее и значительно менее производительна, чем реализация классических крипто алгоритмов типа DES. Обусловлено это сложностью реализации модульной

арифметики (с высокой разрядностью обрабатываемых чисел). Обычно RSA используют для:

1. Шифрования небольших объемов информации;
2. Рассылки классических секретных ключей крипто алгоритмов типа DES;
3. Реализации алгоритмов цифровой подписи.

Для реализации цифровой подписи необходимо, чтобы функции шифрования и дешифрования были коммутативны. Алгоритм RSA обладает таким свойством!

При реализации цифровой подписи **автор документа:**

А) Формирует из текста документа по известному ему и любому получателю документа алгоритму **свертку (хеш-код)** фиксированной длины;

Б) Шифрует свертку **секретным ключом (D,n)** – это ПОДПИСЬ!!!;

В) Передает документ с зашифрованной сверткой получателю (получателям).

Получатель документа, получив документ с зашифрованной сверткой:

А) Формирует из текста документа по известному алгоритму **свертку**;

Б) Дешифрует открытым ключом зашифрованную секретным ключом свертку (**ПОДПИСЬ после дешифрирования**);

В) Сравнивает **свертку** и **ПОДПИСЬ после дешифрирования**, при равенстве – документ оригинален, при неравенстве – документ поддельный (изменен злоумышленником).

Сравнительные характеристики алгоритмов шифрования

Характеристика	DES	RSA
Скорость шифрования	Высокая	Низкая
Используемая функция шифрования	Перестановка и подстановка	Возведение в степень
Длина ключа	56 бит	Более 500 бит
Наименее затратный крипто анализ (его сложность определяет стойкость алгоритма)	Перебор по всему ключевому пространству	Разложение числа на простые множители
Время генерации ключа	Миллисекунды	Минуты
Тип ключа	Симметричный	Асимметричный

ЛЕКЦИЯ 10. МУЛЬТИПРОЦЕССОРНЫЕ ОС

В общем случае различают

- **многопроцессорные вычислительные системы (ВС) с общей (разделяемой) памятью** (вычислители с симметричной мультипроцессорной обработкой – **SMP**),
- **ВС с распределенной памятью – многомашинные системы** (вычислители с массовым параллелизмом – **MPP** и **вычислительные кластеры**),
- **распределенные системы** (слабосвязанные системы), объединяемые компьютерной сетью в единую вычислительную систему на прикладном уровне с помощью промежуточного программного обеспечения, предоставляющего однородный уровень для взаимодействующих приложений.

Для организации мультипроцессорной обработки в данных ВС применяют различные инструменты.

В **SMP**-вычислителях мультипроцессорную обработку организуют многопроцессорные ОС.

В **MPP**-вычислителях используются коммуникационные средства многомашинных систем.

В **распределенных системах** применяется промежуточное программное обеспечение, основанное на документе, файлах, объектах, координации.

МНОГОПРОЦЕССОРНЫЕ ОПЕРАЦИОННЫЕ СИСТЕМЫ (МОС)

МОС нацелены на поддержание высокой производительности вычислительных систем с несколькими процессорами. **Основная задача МОС состоит в обеспечении прозрачности числа процессоров для приложений.** Различают три варианта построения МОС.

Простейшая организация МОС основана на принципе: **каждому процессору – своя ОС.** В данной модели статически разделяется память по числу процессоров, каждый процессор имеет свою копию ОС с реентерабельным разделяемым кодом ОС и индивидуальными данными. При такой организации процессоры работают как независимые вычислители.

Достоинствами такого подхода является

- простота,

– возможность заранее запланированного (статического) совместного использования памяти и внешних устройств,

– возможность взаимодействия приложений через общую область памяти.

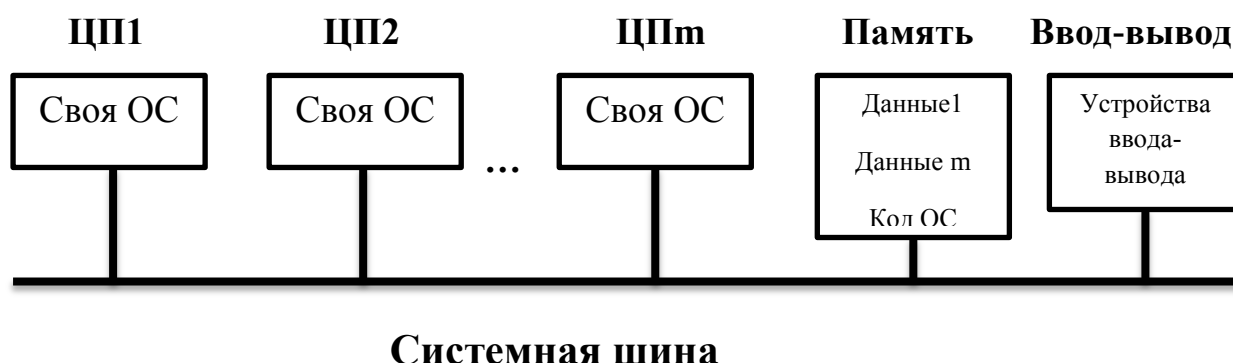
К недостаткам следует отнести:

– дублирование управляющих структур при каждом процессоре,

– невозможность динамического перераспределения используемой памяти,

– невозможность совместного использования ЦП, т.к. каждый процессор планирует свою работу на собственном множестве приложений (как результат – один процессор может быть загружен, а другой – простаивать),

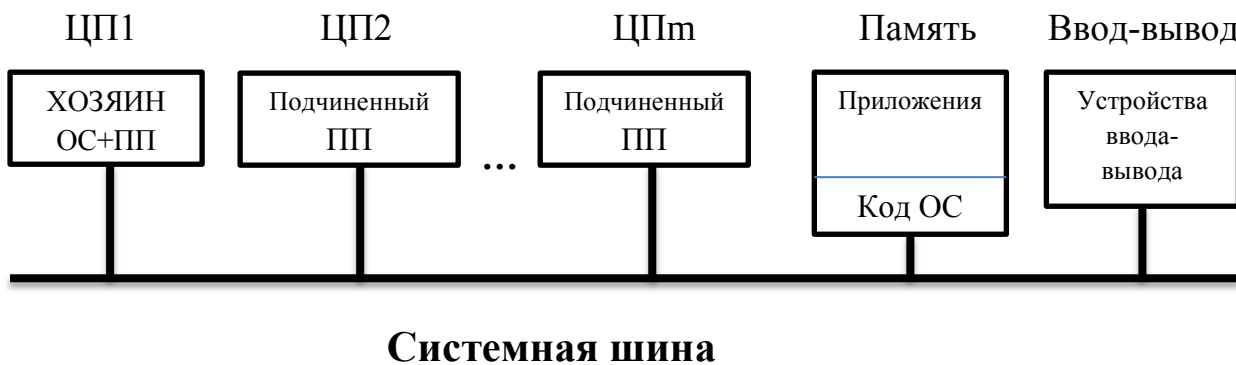
– невозможность обеспечения когерентности кэша данных совместно используемых внешних устройств, поскольку каждый процессор реализует кэширование независимо от других.



В основе второго варианта организации МОС лежит принцип: **хозяин – подчиненный (ведущий – ведомый)**. В данной модели используется одна копия ОС, которая выполняется на первом процессоре вычислителя (ведущий процессор). Все остальные процессоры (ведомые) выполняют прикладные процессы. Все системные вызовы перенаправляются на ведущий процессор, который поддерживает в актуальном состоянии единые управляющие структуры данных. При отсутствии системной работы ведущий процессор также как и остальные выполняет пользовательские приложения.

Основным преимуществом данной организации МОС является отсутствие недостатков предыдущей модели.

Недостатком является потенциально узкое место ведущего процессора при интенсивном потоке системных вызовов от большого числа ведомых процессоров. Такая модель проста и работоспособна при небольшом числе процессоров и применяется обычно во встроенных системах.



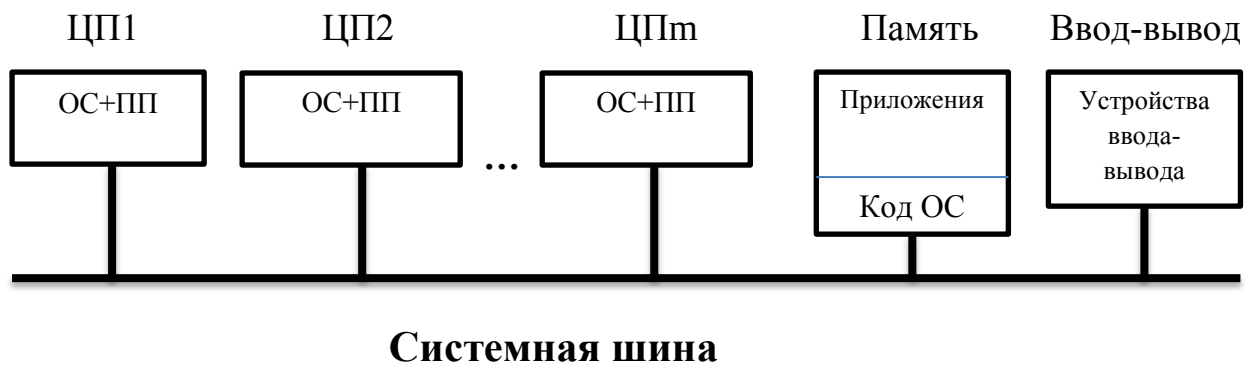
Третья модель МОС основана на **симметричной мультипроцессорной обработке** и устраняет перекося второй модели. Здесь также как и в предыдущей схеме имеется одна копия ОС, но выполнять ее может любой процессор. Системные вызовы от приложений вызывают прерывание на процессоре, выполняющем прикладной процесс с переходом из пользовательского режима в режим ядра и последующей обработкой системного вызова.

В рамках данной модели обеспечивается

- динамический баланс загрузки всех процессоров,
- эффективное использование памяти.

Основной недостаток симметричной МОС состоит в необходимости синхронизации доступа конкурирующих процессоров вычислителя к управляющим структурам.

Для снижения издержек от конкуренции симметричную МОС расщепляют на независимые критические области, не взаимодействующие друг с другом, которые могут выполняться параллельно на различных процессорах. При этом синхронизировать нужно только доступ к конкретным частям МОС. Как следствие код симметричной МОС имеет сложную структуру и требует команды высококвалифицированных специалистов для развития функциональности и сопровождения в течение жизненного цикла. Модель симметричной МОС используется в настоящее время во всех SMP-вычислителях общего назначения.



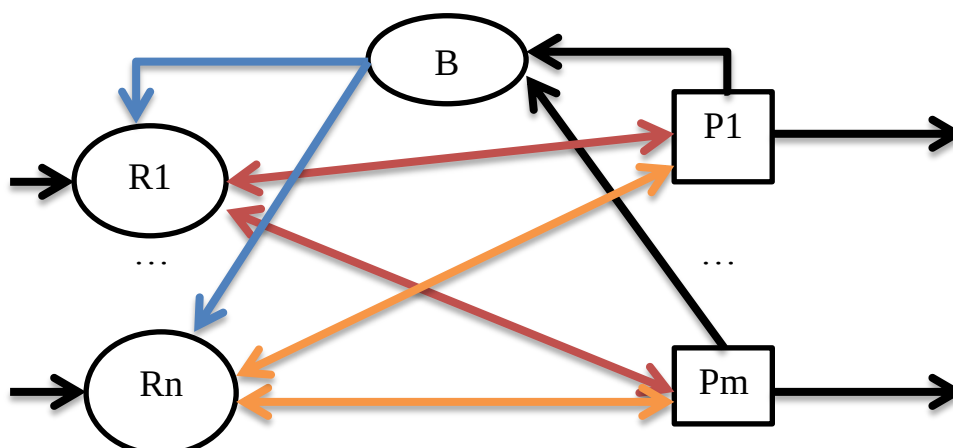
Планирование времени мультипроцессора.

В отличие от однопроцессорной системы, где планирование одномерно, на мультипроцессорном вычислителе планирование двумерно (во времени и в пространстве процессоров). Дополнительным фактором усложняющим планирование является возможная взаимная зависимость связанной группы процессов.

Планирование независимых процессов.

Простейший алгоритм планирования независимых процессов состоит в сопровождении одного или нескольких (по приоритетам) *списков готовых к выполнению процессов*.

- + При этом достигается автоматическая балансировка нагрузки.
- Издержками такого планирования является:
 - высокая конкуренция процессоров за доступ к общим управляющим структурам (прежде всего к очередям готовых процессов),
 - существенные накладные расходы на переключение контекста при блокировании процессов операциями ввода-вывода,
 - значительный объем промахов в кэшах процессоров (и как следствие перемещений блоков памяти и адресных ссылок между кэшем и памятью) при планировании **без предыстории памяти**.

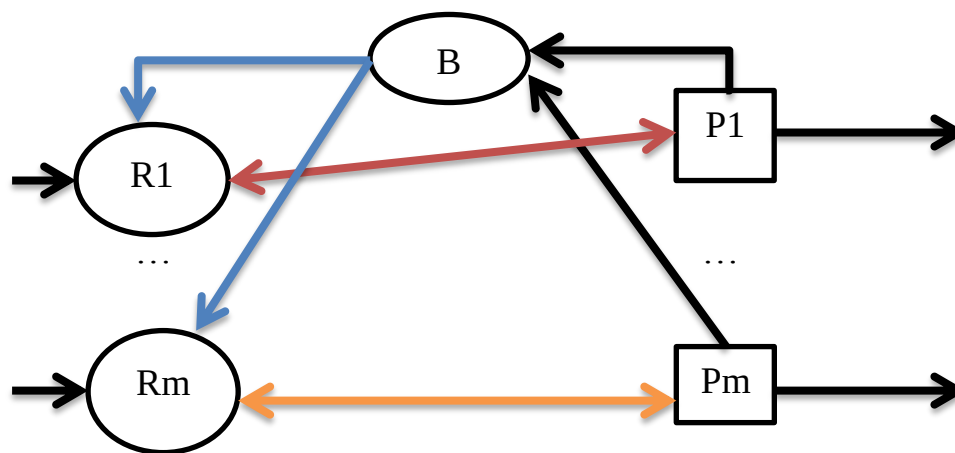


n – приоритетов (очереди готовых процессов), m – процессоров.

Эти недостатки снимаются *родственным планированием* выбора процессов на выполнение. Алгоритм основан на закреплении процессов в течение жизни за определенными процессорами. В момент создания каждый процесс назначается конкретному процессору, при этом **каждый процессор получает свой набор процессов**. Если у какого либо из процессоров отсутствует работа, выполняется перераспределение процессов между процессорами.

Такое планирование в течение жизни процесса обеспечивает

- равномерное распределение нагрузки между процессорами,
- минимизирует конкуренцию за списки процессов (поскольку списки для каждого процессора – свои),
- максимизирует родственность кэша адресных ссылок, а также кэша данных и команд каждого процессора. В силу закрепления процессов за процессорами их кэши, как правило, содержат не вытесненные блоки памяти приложений от предыдущих циклов выделения кванта времени.



n – очередей готовых процессов, m – процессоров.

Планирование времени процессоров для процессов, состоящих из множества связанных потоков.

Простейшим вариантом является *планирование пакетами потоков* только в пространстве процессоров (разделение пространства процессоров). При этом каждый поток выполняется на своем процессоре и удерживает его до завершения потока (в том числе в периоды блокирования потока

операциями ввода-вывода и синхронизации). В каждый момент времени множество процессоров статически разделено на подмножества, в каждом из которых выполняются потоки одного процесса.

Очевидное преимущество такого планирования состоит в устранении многозадачности.

Основным недостатком является существенная потеря времени процессоров при блокировании потоков операциями синхронизации.

Решением данной проблемы является **бригадное планирование** развивающее принцип планирования пакетами потоков на временную ось. При бригадном планировании группы связанных потоков планируются как единое целое – бригада, все члены бригады запускаются одновременно на разных процессорах в режиме разделения времени, все потоки бригады начинают и завершают свои временные интервалы (кванты времени) вместе. В начале каждого кванта все процессоры перепланируются заново, и на каждом процессоре запускается новый поток одной из готовых к выполнению бригад, если какой либо поток блокируется, его процессор простаивает до конца кванта, в течение кванта перепланирование не выполняется. Таким образом, реализуется основная идея бригадного планирования, состоящая в том, чтобы все потоки бригады работали по возможности вместе.

КОММУНИКАЦИОННЫЕ СРЕДСТВА МНОГОМАШИННЫХ СИСТЕМ УРОВНЯ ПОЛЬЗОВАТЕЛЯ.

В системах с массовым параллелизмом наиболее распространены три типа средств: **обмен сообщениями, вызов удаленных процедур, распределенная память совместного доступа**, сохраняющая для программиста модель разработки приложений в едином адресном пространстве разделяемой памяти.

МОДЕЛЬ ПЕРЕДАЧИ СООБЩЕНИЙ построена на парадигме ввода-вывода. Взаимодействие процессов с помощью *обмена сообщениями* организуется, по меньшей мере, двумя операциями (вызовами): *послать* и *принять*. Различают несколько вариантов реализации данных операций.

Первый вариант основан на **блокирующих (синхронных) операциях**, в результате которых процесс, обратившийся к вызову, блокируется до завершения выполнения операции.

Альтернативой блокирующим операциям являются неблокирующие (асинхронные) вызовы, возвращающие управление в вызывающую программу сразу же после запуска операции и обеспечивающие более высокую производительность. Следует отметить, что неблокирующие вызовы приводят к более сложной организации программ.

Неблокирующая операция *послать* может быть реализована тремя способами.

Первое решение основано на копировании отправляемого сообщения в буфер ядра. При этом теряется время процессора на создание дополнительной копии.

Второй вариант предполагает использование прерываний для извещения отправителя о завершении отправки сообщения. В этом случае копирования не требуется, но усложняется организация программы.

Третье решение основано на блокировании «на запись» буфера с отправляемым сообщением до завершения отправки. Данное решение влечет усложнение управления буферным пространством.

Неблокирующий вызов *принять* также имеет три варианта решения.

Первый вариант основан на извещении получателя о получении сообщения с помощью прерываний, что также как и в случае операции *послать* усложняет программу.

Второй вариант предполагает использование почтового ящика для прибывающих сообщений с реализацией процедуры *опроса*, извещающей о поступлении новых сообщений. Основной трудностью при этом является размещение процедуры *опроса* в прикладной программе.

Третье решение основано на активных сообщениях, порождающих временные потоки по обработке прибывающих сообщений. По прибытии сообщения порождается новый (временный) поток в пространстве получающего процесса с указателем на сообщение, который после выполнения заранее определенной процедуры обработки завершается. Основной недостаток решения заключается в значительных накладных расходах на создание и терминирование (завершение) временных потоков.

В модели **КОММУНИКАЦИИ НА ОСНОВЕ ВЫЗОВА УДАЛЕННЫХ ПРОЦЕДУР** вызывающий процесс именуется клиентом, а вызываемый – сервером. По существу взаимодействие процессов в различных узлах многомашинной вычислительной системы сводится к передаче от клиента вызова к серверу и получению от него ответа после исполнения процедуры. Очевидно, что **удаленный вызов процедуры**

должен быть максимально похож на локальный. Однако, несмотря на концептуальную элегантность вызова удаленной процедуры, **локализация клиента и сервера в различных адресных пространствах вносит ряд ограничений.**

Прежде всего, удаленный вызов не допускает передачи параметров через ссылку, а требует явного указания значений параметров. Это может вызвать затруднения в случае необходимости передачи в вызове сложной структуры данных.

Кроме того, в теле процедуры удаленного вызова и клиентского процесса невозможно использование глобальных разделяемых переменных для коммуникации между собой.

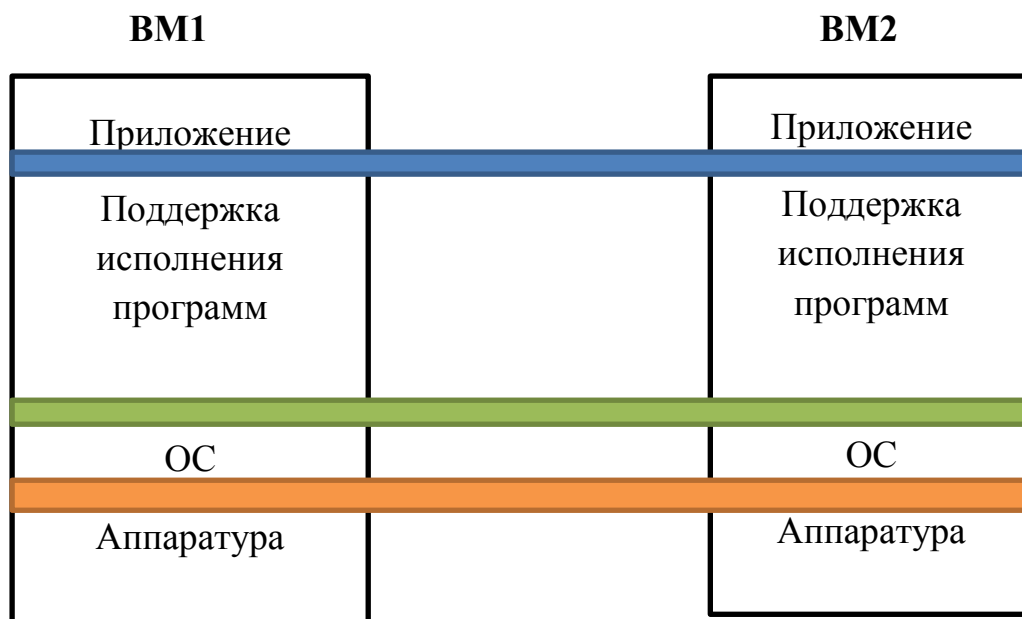
И наконец, реализация удаленного вызова процедур в различных языках программирования требует жесткого мета описания типов и форматов данных, их количества и структуры.

МОДЕЛЬ РАСПРЕДЕЛЕННОЙ ПАМЯТИ СОВМЕСТНОГО ПОЛЬЗОВАНИЯ, реализующая единое адресное пространство, позволяет разрабатывать программы в парадигме общей памяти и переносить на многомашинные платформы традиционные приложения, созданные для SMP-вычислителя. При этом каждый узел вычислителя имеет свою виртуальную память и собственную локальную память.

Отсутствие адресуемого объекта в локальной памяти некоторого узла порождает прерывание, которое удовлетворяется не с локального диска, а по коммуникационной сети с удаленного узла, содержащего данный объект.

Логика организации совместно используемой памяти и единого адресного пространства может быть реализована:

- 1. на аппаратном уровне,**
- 2. на уровне операционной системы,**
- 3. на уровне отдельного, как правило, крупного приложения.**



Основной проблемой данной модели является необходимость обеспечения непротиворечивости копий совместно используемых и изменяемых объектов в различных узлах вычислителя. Реализуется непротиворечивость протоколами когерентности с различными моделями состоятельности.

Поскольку обмен между узлами выполняется блоками памяти значительного размера (для снижения накладных расходов при передаче блоков между узлами), то **проблема непротиворечивости порождает проблему ложного совместного использования** множеством процессов различных несвязанных объектов, расположенных в одном блоке памяти. Очевидно, что чем больше размер блока, тем выше вероятность ложного совместного использования.

КОММУНИКАЦИОННОЕ ПО РАСПРЕДЕЛЕННЫХ СИСТЕМ

Основной целью построения распределенных приложений в распределенных системах (компьютерных сетях) является превращение множества слабосвязанных вычислительных машин в когерентную систему программными средствами. В отличие от многомашинных вычислительных систем узлы территориально распределенной системы:

- связаны существенно слабее (через каналы передачи данных с широким спектром скоростей),
- могут работать на различных технических платформах и под управлением различных ОС,

– могут иметь различные файловые системы (даже под управлением одних и тех же ОС),

– могут иметь различные политики администрирования.

Многомашинные системы включают обычно тысячи узлов, в то время как масштабы распределенной системы могут расширяться до десятков и сотен тысяч узлов.

Сравнение 3-х типов многопроцессорных систем

Аспект	Мультипроцессор	Многомашинная система	Распределенная система
Конфигурация узла	Центральный процессор	Центральный процессор, ОЗУ, сетевой интерфейс	Полноценный компьютер
Периферия узла	Общая, разделяется всеми ЦП	Общая, кроме дисков	Полный набор для каждого узла
Расположение	В одном блоке	В одном помещении	Потенциально по всему миру
Связь между узлами	Общая память	Выделенные линии	Сеть передачи данных
Операционная система	Одна общая	Несколько одинаковых ОС	Могут быть различными
Файловая система	Одна, общая	Одна, общая	У каждого узла своя
Администрирование	Одна организация	Одна организация	Много организаций, у каждого узла своя

С технической точки зрения у мультипроцессоров есть общая память, которой нет у остальных двух многопроцессорных систем. Результатом является использование различных программных моделей. Однако с прикладной точки зрения мультипроцессоры и многомашинные системы представляют собой несколько шкафов с аппаратурой, расположенных в одном помещении. Обе эти системы используются для решения задач, требующих больших объемов вычислений, тогда как распределенная

система, соединяющая компьютеры по интернету, в большей степени занимается связью, чем вычислениями, и используется иначе.

Слабое соединение компьютеров в распределенной системе одновременно является и их силой, и узким местом. **Сильная сторона** состоит в том, что компьютеры могут использоваться для широкого спектра приложений. **Слабость** заключается в том, что программирование таких приложений затрудняется отсутствием лежащей в основе общей модели.

Традиционно в распределенных системах также как и в многомашинных системах можно применять программные парадигмы обмена сообщениями и удаленного вызова процедур. Однако в распределенных системах широкое применение приобрели оригинальные парадигмы построения масштабируемых приложений, основанные на промежуточном программном обеспечении, лежащем над связующим программным обеспечением операционной системы. В целом **распределенные системы стимулировали переход от процессорно-центричной модели компьютера к сетевцентрической модели вычислительных систем.**

Промежуточное ПО, основанное на документе.

Оригинальная парадигма приложения **Всемирная паутина** основана на том, что каждый компьютер может содержать 1 или несколько документов – **web-страниц**. Каждая web-страница может содержать текст, изображения, медиа файлы (звук, видео), и т.д., а также **гиперссылки** (указатели) на др. web-страницы.

Пользовательский интерфейс доступа к web-страницам, рассеянным по узлам сети, реализуется программой – **web-браузером**.

Инструментом доставки web-страниц пользователям обеспечивает протокол **HTTP** (HyperText Transfer Protocol).

Средство создания (разработки) web-страниц реализуется с помощью языка разметки **HTML** (HyperText Markup Language – язык гипертекстовой разметки).

Идентификация web-страниц обеспечивается уникальными сетевыми иерархическими логическими адресами **URL** (Uniform Resource Locator).

Всемирная паутина представляет собой гигантскую коллекцию документов, объединенную с помощью гиперссылок в направленный граф, и работает в парадигме клиент-сервер.

Промежуточное ПО, основанное на файловой системе.

Данный подход состоит в том, чтобы придать распределенной системе вид огромной файловой системы, рассеянной по узлам компьютерной сети. Модель распределенной файловой системы подразумевает, что для связи пользователей (процессов) используется файловый обмен. Один процесс записывает данные в файл, а другой считывает их оттуда. В данной модели возникают следующие проблемы: **модель переноса файлов, именование объектов (файлов), семантика совместного использования файлов.**

МОДЕЛЬ ПЕРЕНОСА ФАЙЛОВ.

Проблема переноса заключается в выборе между **моделью закачивания / скачивания** и **моделью удаленного доступа**.

В **модели закачивания / скачивания** для получения доступа к файлу клиентский процесс сначала полностью считывает его с удаленного сервера (хранящего файл) для обеспечения высокой производительности.

Операции чтения/записи выполняются локально на клиентском компьютере, когда процесс заканчивает работу с файлом, обновленный файл отправляется обратно на сервер.

Преимущество модели:

- простота,
- перенос файла целиком эффективнее, чем перенос его по частям.

Недостатки:

- необходимость наличия достаточно большого объема памяти на клиенте для хранения файла целиком,
- накладные расходы при переносе файла целиком, в то время как требуется только его часть,
- при наличии нескольких конкурирующих пользователей возникает проблема непротиворечивости файлов.

В **модели удаленного доступа** файл остается на сервере, а клиент посылает серверу команды для выполнения работы на самом сервере. Время доступа в этой модели определяется быстродействием сетевого соединения.

ИМЕНОВАНИЕ ОБЪЕКТОВ.

Все распределённые файловые системы поддерживаются каталогами, содержащими файлы. Основной вопрос, возникающий при этом, состоит в том, одинаково ли выглядит иерархия каталогов для всех клиентов.

В распределенных системах различают три общих подхода к именованию файлов и каталогов:

1. **Монтирование** удаленной файловой системы в локальную файловую иерархическую структуру.
2. Имя, задаваемое в виде – **машина+файл**.
3. **Единое пространство имен**, одинаковое на всех машинах распределенной системы.

Именованье файлов в распределенных системах широко опирается на операцию **монтирования**. Системы, работающие с удаленными серверами при помощи монтирования, обладают простотой и гибкостью реализации. Недостатком монтирования является то, что оно не обеспечивает одинаковости вида файловой системы для всех клиентов.

Для обеспечения одинаковости может быть использован глобальный каталог, распознаваемого всеми машинами как корневого. Один из способов поддержки глобального каталога состоит в том, чтобы создать корневой каталог, содержащий только все серверы. При этом все пути к файлам примут одинаковый для всей системы вид **/сервер/путь (машина+файл)**.

Различают две формы прозрачности: **прозрачность местоположения, независимость от местоположения**.

Прозрачность местоположения означает, что по имени пути невозможно определить расположение файла (можно определить сервер, на котором расположен файл, но неизвестно где расположен сам сервер). Сервер может перемещаться по сети без необходимости изменения имени пути.

Независимость от местоположения означает, что файлы могут перемещаться с одного сервера на другой без изменения пути файла. Независимость пути файла от местоположения сложно реализовать, но это свойство очень ценно для распределенных прикладных систем.

Семантика совместного использования файлов.

При использовании файла несколькими пользователями необходимо определить семантику чтения/записи файла для предотвращения возможных проблем. В однопроцессорных системах все операции в/в упорядочиваются естественным образом в единую последовательность, одинаковую для всех процессов. Такая модель обладает **последовательной непротиворечивостью**.

В распределенной системе последовательная непротиворечивость может быть легко достигнута при наличии всего одного сервера. Тогда все операции чтения-записи поступают напрямую на файловый сервер, который обрабатывает их в строгой последовательности. Но один файловый сервер ограничивает производительность. Проблема повышения производительности решается кэшированием (копированием часто используемых файлов в локальной памяти). Но кэширование порождает **проблему когерентности** при совместном использовании одного файла несколькими пользователями.

Возможные решения:

1. **Немедленное распространение всех изменений** кэшируемых файлов обратно на сервер.
2. **Сеансовая семантика**, ослабляющая семантику последовательно непротиворечивого совместного использования файлов. Согласно сеансовой семантики изменения всех открытых файлов сначала видны только процессу, выполнившему эти изменения (хозяину). После закрытия файла изменения видны другим процессам. При использовании и модификации файла одновременно несколькими клиентами необходимо доопределять семантику.
3. **Модель закачивания-скачивания с автоматической блокировкой** скачанных файлов. Попытки всех остальных клиентов получить уже считанный кем-то файл будут приостановлены до того момента, пока первый клиент не вернет файл. При повышенном спросе на файл сервер может посылать клиенту, удерживающему файл, сообщения об ускорении возврата файла.

В целом следует отметить, что составление корректной семантики использования общих файлов является трудной задачей, не имеющей элегантных и эффективных решений.

Промежуточное ПО, основанное на совместно используемых объектах.

В рамках данной парадигмы распределенная система оперирует не с документами и файлами, а с **ОБЪЕКТАМИ**. **Объект – набор переменных, объединенных вместе с набором процедур доступа к ним, называемых методами.** Процессам не разрешается получать доступ к переменным напрямую. Процессы должны для этого вызывать методы.

Примером систем, основанных на объектах, является технология CORBA (Common Object Request Broker Architecture – архитектура распределенных объектных приложений). Технология CORBA представляет

собой систему типа клиент-сервер, в которой клиентский процесс может осуществлять операции с объектами, расположенными на серверах.

Архитектура CORBA разработана для неоднородных аппаратных и программных платформ. Согласование разно платформенных клиента и сервера реализуется через посредников ORB (Object Request Broker – брокер объектных запросов). Каждый объект системы CORBA описывается на специальном языке описания интерфейсов IDL (Interface Definition Language).

В определении объекта указываются методы, экспортируемые объектом, и типы параметров для каждого метода. При создании объекта CORBA создается ссылка на этот объект, которая возвращается процессу, создавшему объект. В дальнейшем процесс обращается к методам созданного объекта по этой ссылке. Ссылка может быть передана другим процессам или сохранена в объектной библиотеке.

Чтобы вызвать метод объекта, клиентский процесс сначала должен получить ссылку на этот объект. Ссылку можно получить либо напрямую от создавшего объект процесса, либо при помощи поиска этого объекта или его метода по имени в специальном каталоге. Когда ссылка на объект получена, клиентский процесс упаковывает параметры вызываемого метода в стандартную структуру и связывается с клиентским ORB-посредником. Тот, в свою очередь, посылает сообщение серверному ORB-посреднику, который вызывает метод объекта. Весь механизм схож с вызовом удаленной процедуры RPC.

Задача ORB-посредников заключается в сокрытии всех низкоуровневых деталей связи и распределения от клиентской и серверной программ. В частности ORB-посредники скрывают от клиента расположение сервера, аппаратное обеспечение ORB сервера, информацию о том, является ли сервер двоичной программой или сценарием, является ли объект активным в настоящий момент, способ общения двух ORB-посредников (TCP/IP, RPC, общая память и т.д.).

Для связи через интернет между клиентским и серверным ORB-посредниками используется протокол IIOP (Internet InterORB Protocol). Для обеспечения возможности использования в системах CORBA объектов, не предназначенных специально для этого, каждый объект оснащается адаптером объекта (упаковщик, обрабатывающий рутинные операции типа формирования ссылок на объект, активация объекта в случае, если тот при вызове находится в неактивном состоянии).

Основной недостаток системы CORBA заключается в том, что каждый объект расположен только на одном сервере, что означает низкую производительность для объектов, одновременно используемых многими клиентами. Обычно архитектура CORBA приемлемо работает только в небольших системах (например, в пределах локальной сети).

Промежуточное ПО, основанное на координации.

Данный вид интерфейсного слоя программного обеспечения для создания приложений основан на парадигме **публикация/подписка**. В рамках этой модели набор процессов, объединенных сетью, может порождать и потреблять информацию.

При появлении у процесса новой информации он широковещательно публикует иерархический каталог ее тем (рассылает ее по сети всем процессам-партнерам). Процессы, которых интересуют определенные темы (читатели), могут подписаться на них. Чтобы подписаться на какую-либо тему, нужно сообщить ее специальному демону-подписчику (подписаться), работающему на машине процесса-подписчика.

При широковещательной рассылке (публикации) издателем сводки нового набора информационных сообщений демон-подписчик на каждой машине селектирует из набора сообщения только на подписанные темы. Модель **публикация/подписка** полностью отделяет (исключает обратную связь) производителей информации (издателей) от потребителей (подписчиков). Очень часто эта модель применяется в современном интернете вещей.

ЛЕКЦИЯ 11 КОГЕРЕНТНОСТЬ МНОГОУРОВНЕВОЙ ПАМЯТИ

Модели состоятельности памяти

Одной из основных проблем масштабируемых многопроцессорных вычислительных систем является построение распределенной разделяемой памяти, имеющей единое адресное пространство и доступ к объектам через операции чтения и записи.

Важнейшая задача при этом состоит в том, как извещать процессоры об изменениях, вызванных выполнением команд записи. Необходимость увеличения быстродействия вычислителей приводит к применению многоуровневой памяти и механизмов отложенной модификации удаленных копий одного и того же экземпляра данных, находящегося в кэшах различных процессоров и оперативной памяти.

При реализации изменений удаленных копий применяется **коммуникационный протокол**, обеспечивающий доставку изменений, и **протокол согласованности** или **когерентности** состояния памяти, предотвращающий использование неизмененных копий модифицированного в одном из процессоров элемента данных.

Для сохранения когерентности данных в соответствии с моделью строгой состоятельности памяти операция чтения из разделяемой памяти должна возвращать последнее записанное значение. Поэтому **при модификации одной из множества копий остальные должны быть либо изменены, либо объявлены несостоятельными** (не соответствующими последнему изменению).

Выполнение этих действий требует реализации средств быстрого распространения изменений, однако в любом случае **следование модели строгой состоятельности** (немедленного изменения значения элемента данных на всех уровнях иерархической памяти) **ведет к снижению производительности**.

Для повышения производительности применяют **модели ослабленной состоятельности памяти**, допускающие появление несогласованных копий данных в ходе их параллельной обработки с последующим обеспечением когерентности копий, при ведущем к тому же результату исполнения приложения, что и при строгой состоятельности.

Любая реализация разделяемой памяти многопроцессорных вычислительных систем характеризуется совокупностью признаков:

- **способ реализации** (аппаратный, программный, аппаратно-программный);
- **тип разделяемых объектов данных** архитектуры процессора и/или операционной системы (слово, кэш-строка, страница, сегмент, фрагмент, объект);
- **модель состоятельности памяти**, определяющая допустимые последовательности доступа в память (SRSW – "один читатель/один писатель", MRSW – "много читателей/один писатель", MRMW – "много читателей/много писателей");
- **политика обеспечения когерентности данных** (модификация, объявление несостоятельными);
- **механизм управления распределением памяти и размещением данных** (централизованный/распределенный, статический/ динамический).

Среди широко используемых в настоящее время моделей состоятельности памяти различают следующие.

1. **Строгая** (strict): каждая операция чтения возвращает последнее записанное значение.
2. **Последовательная** (sequential): все процессоры в системе соблюдают один и тот же порядок выполнения записи и чтения. Процессор, выполняющий запись, приостанавливается до получения подтверждений об объявлении несостоятельными всех копий модифицируемых данных или о модификации этих копий.
3. **Процессорная** (processor): наблюдаемый в двух процессорах порядок выполнения операций чтения-записи может не совпадать, но порядок выполнения записей, производимых каждым процессором, должен быть одним и тем же.
4. **Слабая** (weak): вводит разграничение между обычным и синхронизованным доступом в память при выполнении состоятельности только для обращений к памяти в точках синхронизации различных уровней памяти. Разрешается несогласованность данных вне точек синхронизации.
5. **Свободная** (release): уточняет модель слабой состоятельности и требует состоятельности только между парой операций синхронизации acquire - release. Операция acquire открывает критический интервал и обеспечивает исключительный доступ процессора к разделяемым данным. Операция release закрывает критический интервал и разрешает всем процессорам

доступ к разделяемым данным. При этом операции синхронизации, выполняемые в разных процессорах, должны удовлетворять модели последовательной или процессорной состоятельности памяти.

Свободная модель состоятельности памяти образует целый класс моделей. В ЭТОТ класс входят следующие модификации.

Неторопливо-свободная (lazy release): модель свободной состоятельности, при которой модификация разделяемой памяти откладывается до момента непосредственного обращения к ней. Для данной модели характерно то, что при выполнении операции acquire передаются все предшествующие результаты команд записи, которые исполнялись до наступившего момента синхронизации.

Нетерпеливо-свободная (eager release): модель свободной состоятельности с передачей изменений разделяемой памяти при выполнении операции release, независимо от того, когда эти изменения будут востребованы.

Интервально-свободная (scope): модель свободной состоятельности с разбиением исполнения на глобальные и локальные интервалы гарантирует состоятельность разделяемых данных для всех процессоров только в конце каждого глобального интервала и для последовательности локальных интервалов внутри каждого процессора.

Последовательно-свободная (entry): вариант модели свободной состоятельности, требующий доступа к разделяемым данным, защищённого синхропримитивами. Значения синхронизирующих переменных должны модифицироваться согласно модели последовательной согласованности, при которой все процессоры имеют одну и ту же последовательность обращений к каждой из этих переменных.

Неявная (аппаратная) когерентность

Современные микропроцессоры, применяемые для построения многопроцессорных вычислительных систем, имеют встроенные аппаратные средства обеспечения когерентности данных в иерархической памяти. когерентность кэшей процессоров вычислителя обеспечивается с помощью меж узловых (межмодульных) пересылок.

Размер разделяемой процессорами единицы данных обычно составляет кэш-строку. Методы реализации когерентности определяются видом памяти вычислительной системы (сосредоточенная или распределенная) и типом коммуникационной среды, объединяющей память и процессоры.

Аппаратная реализация когерентности универсальна и не требует наличия в программах дополнительного кода для организации разделяемой памяти. При этом обеспечивается минимальная задержка согласования копий данных и высокая пропускная способность всего механизма когерентности.

СОСРЕДОТОЧЕННАЯ ПАМЯТЬ

Для вычислительных систем с сосредоточенной памятью, реализуемых, как правило, в симметричных многопроцессорных архитектурах, наиболее широкое распространение получил алгоритм поддержки когерентности кэшей **MESI** (Modified, Exclusive, Shared, Invalid). Этот алгоритм обеспечивает когерентность кэш-памяти с обратной записью и минимизирует пересылки данных между основной памятью и кэшем. В общем случае в SMP системах возможна как модификация данных, так и объявление модифицируемых данных несостоятельными.

Алгоритм MESI предполагает:

- вычислитель имеет общую разделяемую память,
- в каждом процессоре содержится локальная кэш-память,
- все процессоры объединены между собой и с основной памятью посредством шины.

Кроме того, к шине подключены внешние устройства. Ключевым в данной конструкции является то, что все действия, выполняемые на шине по инициативе процессоров и внешних устройств с копиями строк в кэшах процессоров и основной памяти, доступны для отслеживания всем процессорам.

При этом в каждый момент времени по шине информация передается одним источником, а воспринимают ее все подключенные к шине абоненты. Если в качестве коммуникационной среды вместо шины используется другой тип соединения, то для работоспособности алгоритма **MESI** необходимо обеспечение указанного "прослушивания" транзакций всеми процессорами вычислительной системы.

При выполнении алгоритма **MESI** каждая строка кэш-памяти процессора может находиться в одном из следующих состояний:

М – строка модифицирована (доступна по чтению и записи только в данном процессоре, поскольку модифицирована командой записи по сравнению со строкой основной памяти);

Е – строка монопольно копированная (доступна по чтению и записи в данном процессоре и в основной памяти);

S – строка множественно копированная или разделяемая (доступна по чтению и записи в данном процессоре, основной памяти и в других процессорах, содержащих ее копию);

I – строка, невозможная к использованию (не доступна ни по чтению, ни по записи).

Состояние строк используется, с одной стороны, для определения процессором возможности локального доступа к кэш-памяти (без выхода на шину), а с другой стороны, для управления механизмом когерентности.

Для управления режимом работы механизма поддержки когерентности применяется бит **WT**, содержащийся в строке КЭШа. Состояние 1 бита **WT** задает режим сквозной записи (write-through), а состояние 0 – режим обратной записи (write-back) в кэш-память.

Динамика состояний строки кэш-памяти, к которой выполняется доступ

Исходное состояние строки	Состояние после чтения	Состояние после записи
М	М	М
Е	Е	М
S	S	Сквозная запись в основную память. Если WT=1 то Е иначе S
I	Если WT=1 то Е иначе S. Обновление строки путем чтения из основной памяти	Сквозная запись в основную память, I

Промех чтения в кэш-памяти вызывает загрузку строки из основной памяти и приведение ее в состояние **Е** или **S**. Заполнение кэш-памяти происходит только при промахх чтения.

В случае промаха записи транзакция записи помещается в буфер и посылается в основную память при предоставлении шины.

При несостоятельной строке в состоянии **I** команда чтения адресуемого объекта из этой строки вызывает чтение строки из основной памяти, размещение ее в кэш-памяти и изменение ее состояния в кэше на **Е** (в режиме сквозной записи) или **S** (в режиме обратной записи).

Состояние **M** не изменяется командами чтения и записи.

В состоянии **E** чтение из строки сохраняет прежнее состояние, а запись переводит строку в состояние **M**.

В состоянии **S** чтение из строки не изменяет ее состояние. Если установлен режим сквозной записи, то после завершения записи состояние строки меняется на **E**. Если же установлен режим обратной записи, то выполняется сквозная запись в основную память, но состояние строки не изменяется.

В состоянии **I** команда записи в строку изменяет только содержимое строки основной памяти, но не изменяет содержимое кэш-памяти и сохраняет состояние строки **I**.

Для поддержки когерентности строк кэш-памяти при выполнении операций ввода-вывода и обращении в основную память других процессоров на шине генерируются специальные циклы опроса состояния кэш-памятей. В рамках этих циклов выясняется хранится ли в кэшах других процессоров строка, содержащая используемый в операции адресуемый объект. Возможен режим принудительного перевода строки в состояние **I** с помощью сигнала **INV**.

Изменение состояний строк под управлением сигнала **INV**

Исходное состояние	Измененное состояние	
	INV=0	INV=1
M	S , обратная запись строки	I , обратная запись строки
E	S	I
S	S	I
I	I	I

РАСПРЕДЕЛЕННАЯ ПАМЯТЬ

Вычислительные системы с памятью, физически распределенной по вычислительным узлам (модулям), но имеющей единое пространство адресов (единую карту памяти), применяют различные алгоритмы реализации когерентности.

Простейший из них заключается в том, что при каждом промахе в кэш:

1. В любом процессоре инициируется запрос требуемой строки из резидентного блока памяти (того блока памяти, в котором эта строка размещена).
2. Запрос передается через коммутатор или другую коммуникационную систему в узел с резидентным для строки блоком памяти.
3. Из резидентного узла необходимая строка пересылается в узел, в котором произошел промах.

В соответствии с такой процедурой обеспечивается, в том числе, начальное заполнение кэшей.

Каждый узел при этом для каждой резидентной строки ведет список узлов, в кэшах которых размещается данная разделяемая узлами вычислителя строка или организуется распределенный по узлам список этих строк.

Для обеспечения когерентности кэшей процессоров после выполнения операции записи процессор приостанавливается до завершения исполнения следующей последовательности действий:

1. Измененная строка кэша пересылается в резидентный блок памяти.
2. Разделяемая строка пересылается из резидентной памяти во все узлы, перечисленные в списке разделяющих эту строку.
3. После получения подтверждений о том, что все копии заменены, резидентный узел пересылает разрешение продолжить вычисления в процессор, приостановленный после записи.

Основным недостатком данного алгоритма является большое время простоя процессоров при выполнении операций записи в кэш. На практике применяются более сложные алгоритмы, обеспечивающие меньшие простои процессоров.

Более совершенным является рассматриваемый ниже алгоритм **DASH**. Каждый блок памяти имеет для каждой строки, резидентной в узле, список

узлов, в кэшах которых размещены копии строк. Каждая резидентная строка может находиться в трех **глобальных** состояниях:

- **NC «не кэшированная»**, если копия строки не находится в кэше какого-либо другого узла, кроме, возможно, резидентного для этой строки;
- **RS «удаленно-разделенная»**, если копии строки размещены в кэшах других узлов;
- **RM «удаленно-измененная»**, если строка изменена операцией записи в каком-либо узле.

Кроме того, каждая строка кэша может находиться в одном из трех **локальных** состояний:

- **NU «невозможная к использованию»**;
- **S «разделяемая»**, если есть неизменная копия, которая, возможно, размещается в других кэшах;
- **M «измененная»**, если копия изменена операцией записи.

ЧТЕНИЕ

Каждый процессор может читать из своего кэша лишь в локальных состояниях читаемой строки «разделяемая» или «измененная». Если строка отсутствует в кэше или находится в состоянии «невозможная к использованию», то посылается запрос «промах чтения», который направляется в узел, резидентный по отношению к требуемой строке.

При глобальном состоянии строки в резидентном узле «некэшированная» или «удаленно-разделенная» копия строки посылается в запросивший узел. Кроме того, список узлов, содержащих копии рассматриваемой строки, пополняется узлом, запросившим копию.

В глобальном состоянии строки «удаленно-измененная» запрос «промах чтения» перенаправляется в узел, содержащий измененную строку. Данный узел пересылает требуемую строку в запросивший и резидентный узлы и устанавливает в резидентном узле для этой строки состояние «удаленно-разделенная».

ЗАПИСЬ

При выполнении операции записи в строку, локальное состояние которой «измененная», вычисления продолжаются сразу же после завершения записи.

В локальных состояниях строки «невозможная к использованию» или «разделяемая» запись в нее начинается с посылки в резидентный для строки узел запроса на захват ее в исключительное использование и приостановки выполнения записи до получения подтверждений о том, что остальные узлы, разделяющие с ним рассматриваемую строку, перевели ее копии в локальное состояние «невозможная к использованию».

Если глобальное состояние строки в резидентном узле «не кэшированная», то строка отсылается запросившему узлу и этот узел продолжает приостановленные вычисления.

Если глобальное состояние строки «удаленно-разделенная», то резидентный узел рассылает по списку всем узлам, имеющим копию строки, запрос на перевод этих строк в локальное состояние "невозможная к использованию". При получении данного запроса каждый из узлов изменяет состояние своей копии строки на «невозможное к использованию» и посылает подтверждение исполнения в узел, инициировавший операцию записи, а в приостановленном резидентном узле строка после исполнения записи переходит в глобальное состояние «удаленно-измененная».

Исходное состояние		Состояние после чтения		Состояние после записи	
Глобальное	Локальное	Глобальное	Локальное	Глобальное	Локальное
NC		RS или NC при чтении в резидентном узле	S	RM/NC	M
RS	NU	RS	S	RM	M
	S				
	M				
RM	NU	RS	S	RM	M
	S				
	M				
ОЗУ	КЭШ	ОЗУ	КЭШ	ОЗУ	КЭШ

Реализация алгоритма когерентности с большим быстродействием возможна за счет учета специфики параллельных программ, в которых асинхронно используются одни и те же переменные на каждом временном интервале только одним процессором с последующим переносом обработки на другой процессор.

Такой тип вычислений позволяет организовать более эффективные схемы передачи необходимых строк из кэша одного процессора в кэш другого.

В вычислительных системах, коммуникационные среды которых основаны на коммутаторах с временным разделением или с пространственным разделением, интерфейс каждого узла с этим коммутатором «прослушивает» все передачи через коммутатор и не ведет списки узлов, разделяющих строки. В коммуникационных средах, построенных на составных распределенных коммутаторах, необходима дополнительная память для ведения и сопровождения списков разделяемых строк. Когерентность обеспечивается при этом хранением информации о состоянии и местонахождении каждой строки разделяемой основной памяти. Элемент такого списка для одной строки должен содержать указанную служебную информацию и ссылки «вперед» и «назад» по списку других строк.

Явная (программная) когерентность

Явная реализация когерентности предусматривает применение программистом специальных команд для работы с локальной памятью и управления контроллерами каналов коммуникационной среды, связывающей узлы вычислительной системы с массовым параллелизмом.

Основная задача разработчика приложения при этом – эффективное программирование параллельных процессов, совмещающих вычисления и передачу данных между узлами и минимизирующих объем передаваемой информации.

Поскольку память в узлах является многоуровневой, то очевидно, что получаемые по коммуникационному интерфейсу строки данных делают несостоятельными копии этих строк в кэш-памяти. Поэтому нужно предусмотреть организацию когерентности прибывшей и кэшированной строк.

Протоколы когерентности памяти при программной реализации являются надстройкой над аппаратными средствами передачи сообщений, что априори создает проблемы с недостаточной пропускной способностью при создании удаленных копий данных, передаче изменений и миграции данных.

Однако программная реализация позволяет учитывать особенности прикладных программ и применять широкое разнообразие протоколов согласованности данных. Различают следующие варианты поддержки когерентности:

1. При наличии дубликатов признаков (тегов) строк кэш-памяти в контроллере прямого доступа делать по получении строки несостоятельной только действительно необходимую строку.
2. При отсутствии признаков памяти в контроллере делать по каждому приему строки несостоятельными все строки кэш-памяти.

Применение явной когерентности в создаваемых вычислительных системах обусловлено либо слишком большим временем, либо недопустимо высокими аппаратными затратами на реализацию неявного механизма когерентности.

Причина этого состоит в высокой сложности применения внутреннего механизма когерентности иерархической памяти узла по сравнению с

использованием предусмотренного для работы с внешними устройствами механизма когерентности.

Как правило, элементами разделяемой памяти при программной реализации когерентности являются страницы, что обусловлено использованием аппаратных средств организации виртуальной памяти для обнаружения операций записи в разделяемые страницы.

Однако относительно большой размер разделяемых страниц создает проблему ложного разделения, при котором конфликтными признаются модификации разными процессорами различных ячеек памяти одной страницы, не конфликтующие между собой. Решением данной проблемы может быть либо введение логических разделяемых переменных, объектов или типов данных, либо отображение множества логических страниц на одну физическую.

Механизм когерентности на основе масштабируемого когерентного интерфейса SCI

Масштабируемый когерентный интерфейс (Scalable Coheret Interface) принят в качестве стандарта ANSI. SCI предусматривает реализацию когерентности на основе стандартно организованной кэш-памяти дополнительного уровня, размещаемой в узле вычислительной системы.

Поиск данных в кэше узла производится при промахе в предыдущих уровнях иерархии памяти. При наличии данных в кэше узла и их состоятельности копия данных пересылается в верхние уровни памяти. Если данные находятся в кэше узла в модифицируемом состоянии, то ожидается окончание модификации, после чего копия данных перемещается выше по иерархии.

При отсутствии данных в кэше узла вырабатывается сигнал промаха, который активизирует контроллер интерфейса на действия по доставке данных из удаленных блоков памяти. Реализация узлов на основе SCI может поддерживать не все возможности SCI (не использовать кэш-памяти и протокола когерентности). В этом случае узлы SCI используются для построения коммуникационной среды со стандартным способом адресации, форматом и протоколом передачи пакетов.

Стандарт SCI позволяет строить коммуникационные среды для объединения узлов в сосредоточенные и распределенные вычислительные системы с быстродействием от 1 Гбит/с до 1 Гбайт/с и более.

Коммуникационная среда SCI предусматривает в каждом узле наличие входного и выходного каналов. Узлы связываются между собой или с коммутатором однонаправленными каналами "точка-точка". При объединении узлов должна формироваться циклическая магистраль (кольцо). Один из узлов в кольце выполняет функции инициализации узлов Кольца с установлением адресов, управления таймерами, уничтожения пакетов, не нашедших адресата. Данный узел, называемый scrubber и являющийся единственным в кольце, помечает проходящие через него пакеты и уничтожает уже помеченные. С помощью SCI могут быть реализованы различные структуры меж соединений: кольцо, тор, решетка.

Информационный обмен между узлами SCI реализуется с помощью транзакций чтения, записи, блокировки, когерентности строк кэш-памяти и основной памяти, передачи сообщений и сигналов прерываний. Все транзакции используют пересылку SCI пакетов между узлом-источником и узлом-получателем. Транзакции предусматривают передачу пакетов с блоками данных фиксированной длины 64 и 256 байтов и переменной длины от 1 до 16 байтов. Фиксированная длина пакетов упрощает логику и увеличивает быстродействие интерфейсов.

Интерфейсом SCI предусматривается 64-разрядная архитектура с 64-битным адресным пространством. Старшие 16 бит определяют адрес узла и применяются для маршрутизации SCI пакетов между узлами вычислительной системы. Остальные 48 битов адресуют внутреннюю память узла. Стандартом SCI предусматривается поддержка отображения и обмена между уровнями памяти блоками по 64 байта.

Протокол когерентности кэш-памятей определяет правила работы с множеством локальных копий одних и тех же данных. При аппаратной реализации этой функциональной возможности SCI снижается сложность операционной системы и системы программирования. В SCI протокол когерентности основан на распределенных директориях (каталогах). Все узлы с копиями данных участвуют в распределенном списке узлов, использующих одну и ту же строку данных. Признак каждой кэш строки содержит указатели на предыдущий и следующий узлы, разделяющие эту строку (имеющие копии строки в своей кэш-памяти). **Головной узел** содержит указатель на контроллер банка памяти, содержащего разделяемую строку, который, в свою очередь, имеет указатель на головной узел. **Оконечный узел** не содержит указателя на последующий узел. При этом каждая разделяемая строка памяти связана с распределенным списком узлов,

разделяющих эту строку. Все узлы, разделяющие строку, участвуют в формировании этого списка.

В зависимости от архитектуры процессора и исполняемой прикладной программы различают

- **когерентное кэширование,**
- **некогерентное кэширование,**
- **не кэширование строк памяти.**

При когерентном обращении узла к памяти за строкой контроллер памяти сохраняет адрес этого узла и возвращает адрес предыдущего узла, запрашивавшего эту строку. Если данные в памяти состоятельные (никакой узел не запрашивал доступа для изменения содержимого строки), то данные из памяти сразу передаются запросившему их узлу. Если строка памяти не состоятельна, а состоятельные данные находятся в каком-либо кэше, то возвращается только адрес узла, запрашивавшего эту строку последним. С помощью данного адреса запрашивающий узел обращается непосредственно в узел, имеющий состоятельные данные, после чего завершается формирование списка узлов, разделяющих строку.

Согласно протоколу доступ по записи в строку памяти представлен только узлу в начале списка. Для получения доступа по записи узел, бывший элементом списка, удаляет себя из списка и запрашивает доступ к памяти с правом записывать данные. В результате узел становится в начале списка и получает право модифицировать строку, но при этом обязан отметить как несостоятельные (очистить) все последующие элементы списка.

Стандартом предусмотрена сильная и слабая поддержка последовательного выполнения кода программ. При **слабой** поддержке операция записи в строку может быть произведена прежде, чем завершится очистка списка. При **сильной** поддержке последовательного выполнения программного кода запись должна производиться только после очистки списка. SCI поддерживает и линейное, и древовидное распространение очистки списка.

Предусматривается также возможность обмена двум узлам данными, находящимися в кэше, без обращения к памяти до тех пор, пока некий третий узел не запросит эти данные.

Основное достоинство протокола когерентности, используемого в SCI, состоит в том, что при расширении системы не нужно ничего менять в уже существующем вычислителе.

Недостатком является необходимость дополнительной памяти для хранения указателей списка.

Структура узла SCI предусматривает одновременную возможность отсылки сформированных в узле пакетов, приема пакетов, адресованных узлу, и ретрансляции транзитных пакетов. Для реализации указанных функций в узле имеется **три соответствующих FIFO очереди: выходная, входная и проходная**. Управление передачей пакетов между узлом-получателем и узлом-отправителем осуществляется в соответствии с протоколом, основанном на процедуре с решающей обратной связью. Размер окна управляющей процедуры равен 64.

ЛЕКЦИЯ 12 ВИРТУАЛИЗАЦИЯ

Средства виртуализации

Виртуализация – предоставление набора ресурсов вычислительной системы или их логического объединения, абстрагированное от аппаратной реализации, и обеспечивающее при этом логическую изоляцию вычислительных процессов друг от друга, выполняемых на одном физическом ресурсе.

В качестве **объекта виртуализации** может выступать

1. Хранилище данных.
2. Приложение.
3. Вычислительная система.
4. Сеть передачи данных.

Средством виртуализации выступает программное обеспечение, которое помогает сделать вычислительные среды независимыми от физической инфраструктуры.

Наиболее ярким примером применения виртуализации является виртуализация вычислительных систем, обеспечивающая разбиение физического сервера на несколько полностью независимых виртуальных машин (ВМ) и предоставляющая возможность запуска нескольких операционных систем на одном компьютере (узле).

При этом каждый из экземпляров таких гостевых операционных систем работает со своим набором динамически выделяемых логических ресурсов (процессоры, область оперативной памяти, устройства хранения), предоставлением которых из общего пула, доступного на уровне оборудования, управляет хостовая операционная система – **гипервизор**.

Гипервизор напрямую взаимодействует с физическими ресурсами вычислителя и отвечает за то, чтобы виртуальные машины «видели» эти ресурсы как собственные. Гипервизор может в разных пропорциях распределять между ОС физические ресурсы вычислителя. Такой вид виртуализации вычислительных систем требует специальной аппаратной поддержки.

Независимая ВМ образует изолированный контейнер программного обеспечения, содержащий операционную систему и приложения. Виртуализация на уровне операционной системы обеспечивает работу нескольких экземпляров пространства пользователя в пределах одной ОС.

К достоинству виртуализации вычислительных систем следует отнести

1. Изоляцию неисправностей и нарушений системы безопасности на аппаратном уровне.
2. Сохранение уровня производительности с помощью расширенных средств управления ресурсами.
3. Сохранение состояния виртуальной машины полностью в виде файлов, дающее возможность перемещения и копирования виртуальных машин операциями с файлами.
4. Независимость ВМ от оборудования (при наличии гипервизора для данной архитектуры вычислителя) – обеспечивает простоту инициализации и миграции виртуальных машин на любой сервер.
5. Виртуализация повышает адаптивность, гибкость, масштабируемость ИТ-среды, производительность и доступность ресурсов за счет кластеризации серверов.
6. Снижает эксплуатационные расходы, минимизирует простои, существенно упрощает администрирование инфраструктуры и лицензирование программного обеспечения.
7. Обычно ВМ поддерживают технологию моментальных снимков и значительно облегчают резервное копирование и восстановление виртуальных систем.

Примерами наиболее развитых функциональных решений для виртуализации серверов являются **Microsoft Hyper-V** и **VMware vSphere**, которые требуют поддержки процессором аппаратных технологий виртуализации **Intel VT** (VT-x, Intel Virtualization Technology for x86) или **AMD-V**. Аппаратная поддержка виртуализации позволяет реализовать в многопроцессорных вычислительных системах так называемую доменную архитектуру.

Доменная архитектура основана на концепции разбиения SMP-вычислителей, предусматривающей сегментирование ресурсов вычислительной системы и создание изолированных друг от друга разделов, обеспечивающих независимую работу разных приложений.

Данная архитектура позволяет:

1. Объединить в одном вычислителе несколько полнофункциональных независимых серверных систем, каждая из которых имеет свои процессоры, память, подсистему ввода-вывода и экземпляр операционной системы.
2. Изолировать ошибки в приложениях, исполняющихся в различных разделах (они не влияют на работу прикладных систем в других разделах).

3. Исключить конкуренцию между приложениями за доступ к ресурсам вычислительной системы.
4. Обеспечить высокую масштабируемость.
5. Упростить управление совокупностью серверов.
6. Сделать прикладную среду более безопасной, изолируя аппаратные сбои и программные ошибки в разделе.
7. Снизить затраты на аппаратуру.

В современных вычислительных системах с SMP-архитектурой реализуют два типа разбиений – системное и разбиение на уровне ресурсов приложений.

Системные разделы. Системное разбиение позволяет создать на базе одного аппаратного сервера несколько разделов, каждый из которых представляет собой полнофункциональную серверную систему, работающую под управлением собственного экземпляра ОС.

Системное разбиение является необходимой функциональностью для провайдеров приложений.

Системный раздел включает все необходимые для автономной работы ОС ресурсы процессоры, память, устройства ввода-вывода.

Это позволяет одновременно

1. Выполнять эксплуатацию приложений.
2. Тестировать новые версии приложений.
3. Выполнять миграцию к новым версиям ОС и приложений.
4. Объединять в одном сервере стадии выполнения, разработки и тестирования разных приложений.

Системное разбиение является эффективным механизмом изоляции критичных корпоративных приложений, решающих задачи различных подразделений.

Кластеризация системных разделов позволяет существенно повысить надежность вычислителя, обеспечить автоматическую балансировку нагрузки с помощью динамического перераспределения клиентских соединений.

Различают статическое и динамическое разбиение.

Статическое разбиение определяет системные ресурсы, входящие в раздел (процессоры, память, шины ввода-вывода, внешние устройства), до запуска ОС, а их конфигурация не может быть изменена в ходе работы.

Динамическое разбиение позволяет создавать и удалять разделы, менять состав их ресурсов в оперативном режиме. Динамическое разбиение обеспечивает динамическое управление рабочей нагрузкой, так как конфигурацию системы можно менять в зависимости от создавшейся ситуации, передавая приложениям в период пиковых нагрузок дополнительное число процессоров.

Разделение приложений. Разделение приложений является удобным механизмом управления ресурсами вычислительной системы при предоставлении информационных услуг. Возможность закрепления за конкретным приложением или группой пользователей необходимых ресурсов упрощает достижение соглашений об уровне обслуживания (QoS).

При разделении приложений наборы ресурсов для рабочих областей приложений, контролируемых одним экземпляром ОС, выделяются в "прикладные разделы". Разделение приложений дополняет системное разбиение и допустимо только в определенных аппаратных границах: на одном автономном сервере, в серверном кластере, в одном системном разделе.

Планирование классов. Следующий уровень программного разбиения обеспечивает "**планирование классов**" – сегментирование ресурсов прикладного раздела на несколько прикладных или пользовательских классов и явное выделение части ресурсов группе пользователей или приложений, входящих в класс.

В основе разделения приложений лежит выделение некоторого числа процессорных блоков на сервере или системном разделе в **процессорный набор**. Каждый процессорный набор относится к одному прикладному разделу. В результате приложения и пользователи, которым выделен раздел с конкретным процессорным набором, не смогут воспользоваться ресурсами прикладного раздела с другим процессорным набором, что исключает взаимное влияние на производительность приложений, исполняемых в различных разделах. Обычно при этом процессорные наборы способны изменяться динамически за счет перемещения процессоров между наборами.

Кроме выделения процессорных наборов возможно введение ограничений на объем виртуальной или физической памяти, используемой в данном прикладном разделе. **Дополнительной поддержкой разделения приложений являются средства управления пропускной способностью, ограничивающие сетевой трафик приложения на уровне порта, и**

установление квот на дисковое пространство, используемое для прикладного раздела или класса. При этом ограничения могут быть программными или аппаратными. Для ограничений, вводимых программно, системой выдаются предупреждения о превышении квоты, а при аппаратных ограничениях запрещается запись в файлы.

Разбиение на прикладные разделы позволяет управлять приложением в случае, когда оно расходует слишком много процессорного времени или использует большой объем оперативной памяти. Разделение приложений предоставляет возможность гибкого управления ресурсами приложений для различных задач. Разбиение на уровне приложений позволяет повысить их производительность, поскольку снижает вероятность конфликтов за ресурсы.

В результате появляется возможность перераспределения нагрузки системы путем переназначения ресурсов между коллективами пользователей или группами задач для балансировки рабочей нагрузки. Это делает более предсказуемым поведение приложений, что в свою очередь повышает качество информационного обслуживания и доступность приложений.